

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN HỆ CƠ SỞ DỮ LIỆU

Tài liệu dịch chương 6

Giảng viên hướng dẫn: Đào Thị Ngọc Hân

STT nhóm: 11 (lớp 67IT4)

Sinh viên thực hiện: Hoàng Văn Đức (nhóm trưởng)

Nguyễn Quang Nghĩa

Nguyễn Văn Kiên

Nguyễn Mạnh Cường

Hà Nội, 10-2023

Chương 6: Normalization of Database Tables

(Chuẩn hóa của các bảng cơ sở dữ liệu)

Sau khi hoàn thành chương này, bạn sẽ có thể:

- Giải thích quá trình chuẩn hóa và vai trò của nó trong quá trình thiết kế cơ sở dữ liệu
- Xác định và mô tả từng dạng chuẩn: 1NF, 2NF, 3NF, BCNF và 4NF
- Giải thích cách chuyển đổi từ dạng chuẩn thấp hơn sang dạng chuẩn cao hơn
- Áp dụng các quy tắc chuẩn hóa để đánh giá và sửa đổi cấu trúc bảng
- Xác định các tình huống đòi hỏi thực hiện đồng bộ hóa để tạo thông tin một cách hiệu quả
- Sử dụng danh sách kiểm tra mô hình dữ liệu để kiểm tra ERD đáp ứng một số yêu cầu tối thiểu.

Preview:

Thiết kế cơ sở dữ liệu tốt phải phù hợp với cấu trúc bảng tốt. Trong chương này, bạn sẽ học cách đánh giá và thiết kế cấu trúc bảng tốt để kiểm soát sự trùng lặp dữ liệu, tránh các hiện tượng dữ liệu bất thường. Quá trình đưa ra các kết quả mong muốn như vậy được gọi là chuẩn hóa.

Để nhận ra và đánh giá các đặc điểm của một cấu trúc bảng tốt, có ích để xem xét một cấu trúc bảng kém chất lượng và các vấn đề nó tạo ra. Sau đó, bạn sẽ học cách sửa cấu trúc bảng này. Phương pháp này sẽ mang lại những lợi ích quan trọng: bạn sẽ biết cách thiết kế một cấu trúc bảng tốt và cách sửa chữa một cấu trúc bảng kém chất lượng.

Bạn sẽ khám phá không chỉ rằng các hiện tượng dữ liệu bất thường có thể loại bỏ thông qua chuẩn hóa mà cả một bộ cấu trúc bảng đã được chuẩn hóa đúng cách thực tế có thể ít phức tạp hơn so với bộ cấu trúc bảng chưa được chuẩn hóa. Ngoài ra, bạn sẽ học được rằng bộ cấu trúc bảng đã được chuẩn hóa đầy đủ thực sự phản ánh chính xác hơn hoạt động thực tế của một tổ chức.

Data Files and Available Formats

	MS Access	Oracle	MS SQL	My SQL		MS Access	Oracle	MS SQL	My SQL
CH06_ConstructCo	✓	✓	✓	✓	CH06_Service	✓	✓	✓	✓
CH06_Eval	✓	✓	✓	✓					

Data Files Available on cengagebrain.com

6-1. Bảng cơ sở dữ liệu và chuẩn hóa

normalization (Chuẩn hóa)

Một quy trình gán các thuộc tính cho các thực thể để giảm hoặc loại bỏ sự dư thừa dữ liệu.

denormalization (Sự không chuẩn hóa)

Một quá trình trong đó một bảng được thay đổi từ dạng chuẩn cấp cao hơn sang dạng chuẩn cấp độ thấp hơn, thường là để tăng tốc độ xử lý. Việc không chuẩn hóa có khả năng mang lại sự bất thường về dữ liệu.

prime attribute (thuộc tính chính)

Một thuộc tính quan trọng; nghĩa là một thuộc tính là một phần của khóa hoặc là toàn bộ khóa. Xem thêm các thuộc tính khóa.

key attributes (thuộc tính khóa)

Các thuộc tính tạo thành khóa chính. Xem thêm thuộc tính chính.

Nonprime attribute (thuộc tính không chính)

Một thuộc tính không phải là một phần của khóa.

nonkey attribute (thuộc tính không khóa)

Xem thuộc tính không chính thức

Chương trình quản lý cơ sở dữ liệu quan hệ tốt không đủ để tránh sự trùng lặp dữ liệu được thảo luận trong Chương 1, Hệ thống cơ sở dữ liệu (Chapter 1, Database Systems). Nếu các bảng cơ sở dữ liệu được thiết kế như các tệp trong hệ thống tệp, hệ thống quản lý cơ sở dữ liệu quan hệ (the relational database management system - RDBMS) không có cơ hội thể hiện khả năng xử lý dữ liệu chuyên nghiệp của nó.

Bảng là khối xây dựng cơ bản của thiết kế cơ sở dữ liệu. Do đó, cấu trúc của bảng là rất quan trọng. Lý tưởng nhất, quá trình thiết kế cơ sở dữ liệu được khám phá trong Chương 4, Mô hình Thực thể Quan hệ (Entity Relationship (ER) Modeling), sẽ dẫn đến các cấu trúc bảng tốt. Tuy nhiên, có thể tạo ra các cấu trúc bảng kém chất lượng, ngay cả trong thiết kế cơ sở dữ liệu tốt. Làm thế nào để nhận biết một cấu trúc bảng kém chất lượng, và làm thế nào để tạo ra một bảng tốt? Câu trả lời cho cả hai câu hỏi này liên quan đến chuẩn hóa.

Normalization (Chuẩn hóa) là quá trình đánh giá và sửa đổi cấu trúc bảng để giảm thiểu sự trùng lặp dữ liệu và giảm thiểu khả năng có những lỗi dữ liệu không mong muốn. Quá trình chuẩn hóa bao gồm gán các thuộc tính cho các bảng dựa trên các khái niệm của quyết định và phụ thuộc chức năng bạn học ở Chương 3, Mô hình Cơ sở Dữ liệu Quan hệ (Chapter 3, The Relational Database Model).

Quá trình chuẩn hóa thực hiện thông qua một loạt các giai đoạn được gọi là các dạng chuẩn. Ba giai đoạn đầu tiên được mô tả là dạng chuẩn đơn vị đầu tiên (1NF), dạng chuẩn đơn vị thứ hai (2NF) và dạng chuẩn đơn vị thứ ba (3NF). Từ quan điểm cấu trúc, hình thức 2NF tốt hơn hình thức 1NF và hình thức 3NF tốt hơn hình thức 2NF. Đối với hầu hết các mục đích trong thiết kế nghiệp vụ cơ sở dữ liệu, 3NF là cao nhất mà bạn cần phải đạt được trong quá trình chuẩn hóa. Tuy nhiên, bạn sẽ khám phá ra rằng các cấu trúc bảng 3NF được thiết kế đúng cách thực sự đáp ứng các yêu cầu của dạng chuẩn thứ tư (4NF).

Mặc dù chuẩn hóa là một thành phần rất quan trọng trong thiết kế cơ sở dữ liệu, bạn không nên cho rằng dạng chuẩn cao nhất luôn là tốt nhất. Nói chung, dạng chuẩn càng cao thì bạn cần phải sử dụng nhiều phép nối để tạo ra đầu ra cụ thể hơn. Ngoài ra, hệ thống cơ sở dữ liệu cũng cần sử dụng nhiều tài nguyên để đáp ứng các truy vấn của người dùng. Một thiết kế thành công cũng phải xem xét nhu cầu của người dùng cuối cùng để đạt được hiệu suất nhanh. Vì vậy, đôi khi bạn cần phải không chuẩn hóa một số phần của thiết kế cơ sở dữ liệu để đáp ứng yêu cầu hiệu suất. **Denormalization** (Sự không

chuẩn hóa) sẽ dẫn đến sự trùng lặp dữ liệu nhiều hơn, đồng thời giảm dạng chuẩn; tức là một cấu trúc bảng 3NF sẽ được chuyển đổi thành 2NF thông qua việc không chuẩn hóa. Tuy nhiên, giá phải trả cho việc tăng hiệu suất qua việc không chuẩn hóa là sự trùng lặp dữ liệu nhiều hơn.

Lưu ý:

Mặc dù từ *table* (bảng) được sử dụng xuyên suốt chương này, nhưng về mặt hình thức, chuẩn hóa có liên quan đến các mối quan hệ. Trong Chương 3, bạn đã biết rằng các thuật ngữ *table* và *relation* (quan hệ) thường được sử dụng thay thế cho nhau. Trong thực tế, bạn có thể nói rằng bảng là khung nhìn triển khai của một mối quan hệ logic đáp ứng một số điều kiện cụ thể. (Xem Bảng 3.1.) Tuy nhiên, chặt chẽ hơn, quan hệ toán học không cho phép các bộ dữ liệu trùng lặp; trong khi chúng có thể tồn tại trong bảng (xem Phần 6-5). Ngoài ra, trong thuật ngữ chuẩn hóa, bất kỳ thuộc tính nào ít nhất là một phần của khóa đều được gọi là **prime attribute** (thuộc tính chính) thay vì **key attribute** (thuộc tính khóa) thuật ngữ phổ biến hơn đã được giới thiệu trước đó. Ngược lại, **nonprime attribute** (thuộc tính không chính) hoặc **nonkey attribute** (thuộc tính không khóa) không phải là một phần của bất kỳ khóa ứng cử viên nào.

6-2. Sự cần thiết của việc chuẩn hóa

Chuẩn hóa thường được sử dụng cùng với mô hình thực thể quan hệ mà bạn đã học ở các chương trước. Các nhà thiết kế cơ sở dữ liệu thường sử dụng chuẩn hóa trong hai trường hợp. Khi thiết kế cấu trúc cơ sở dữ liệu mới dựa trên yêu cầu nghiệp vụ của người dùng cuối, người thiết kế cơ sở dữ liệu có thể xây dựng mô hình dữ liệu bằng cách sử dụng một kỹ thuật như ERD ký hiệu Crow's Foot. Sau khi thiết kế ban đầu hoàn tất, người thiết kế có thể sử dụng chuẩn hóa để phân tích mối quan hệ giữa các thuộc tính trong mỗi thực thể và xác định xem cấu trúc có thể được cải thiện thông qua chuẩn hóa hay không. Ngoài ra, và cũng thường xuyên hơn, các nhà thiết kế cơ sở dữ liệu thường được yêu cầu sửa đổi cấu trúc dữ liệu hiện có có thể ở dạng tệp phẳng, bảng tính hoặc cấu trúc cơ sở dữ liệu cũ hơn. Một lần nữa, bằng cách phân tích mối quan hệ giữa các thuộc tính hoặc trường trong cấu trúc dữ liệu, người thiết kế cơ sở dữ liệu có thể sử dụng quy trình chuẩn hóa để cải thiện cấu trúc dữ liệu hiện có và tạo ra thiết kế cơ sở dữ liệu phù hợp. Cho dù bạn đang thiết kế cấu trúc cơ sở dữ liệu mới hay sửa đổi cấu trúc hiện có thì quy trình chuẩn hóa đều giống nhau.

Rất hiếm khi thiết kế một cơ sở dữ liệu hoàn toàn mới chỉ bằng cách chuẩn hóa. Thông thường, bạn bắt đầu bằng việc xác định các quy tắc công việc và ràng buộc dữ liệu, xác định các phân phụ thuộc chức năng, các thực thể và thuộc tính bằng cách sử dụng các kỹ thuật bạn đã học trong các chương trước. Sau đó, bạn áp dụng các khái niệm chuẩn hóa để xác thực và tinh chỉnh thêm mô hình.

Chương này là một trong những chương quan trọng nhất trong cuốn sách vì ở đây bạn sẽ tìm hiểu cách các khái niệm bạn đã học trước đó phối hợp với nhau trong thiết kế cơ sở

dữ liệu để lập mô hình cơ sở dữ liệu đáp ứng các ràng buộc về tính toàn vẹn dữ liệu cũng như các yêu cầu về hiệu suất và báo cáo của người dùng. Những khái niệm này bao gồm:

- Xác định các quy tắc nghiệp vụ
- Xác định và định nghĩa các hạn chế về nghiệp vụ và dữ liệu
- Xác định sự phụ thuộc chức năng
- Nhận dạng các thực thể và mối quan hệ
- Loại bỏ các thuộc tính đa giá trị

Mục tiêu chính của việc chuẩn hóa là loại bỏ các dị thường về dữ liệu bằng cách loại bỏ các phần dư thừa dữ liệu không cần thiết hoặc không mong muốn. Để đảm bảo các mục tiêu đã nêu trước đây của thiết kế cơ sở dữ liệu, quá trình chuẩn hóa sử dụng khái niệm phụ thuộc hàm để xác định thuộc tính nào (hoặc tập hợp thuộc tính) xác định các thuộc tính khác. Hãy ghi nhớ điều này khi bạn làm việc thông qua các ví dụ.

Để hiểu rõ hơn về quy trình chuẩn hóa, hãy xem xét các hoạt động báo cáo được đơn giản hóa của một công ty xây dựng quản lý một số dự án xây dựng. Mỗi dự án có mã số dự án, tên, nhân viên được phân công, v.v. Mỗi nhân viên có mã số nhân viên, tên và phân loại công việc, chẳng hạn như kỹ sư hoặc kỹ thuật viên máy tính.

Công ty tính phí khách hàng bằng cách thanh toán số giờ dành cho mỗi hợp đồng. Tỷ lệ thanh toán theo giờ phụ thuộc vào phân loại công việc của nhân viên. Ví dụ: một giờ thời gian của kỹ thuật viên máy tính được tính phí ở mức khác với một giờ thời gian của kỹ sư. Định kỳ, một báo cáo dự án được tạo ra có chứa thông tin được hiển thị trong Bảng 6.1. Báo cáo này sắp xếp dữ liệu cho từng dự án thành một định dạng tóm tắt.

Trong trường hợp này, nhà tư vấn có nhiệm vụ tạo cơ sở dữ liệu để hỗ trợ kịch bản báo cáo này. Bước đầu tiên là tập trung vào dữ liệu cơ sở cần thiết để tạo báo cáo. Tổng phí, tổng phụ và tổng đều là dữ liệu phái sinh. Nhớ lại Chương 4 rằng dữ liệu dẫn xuất có thể được lưu trữ hoặc không trong cơ sở dữ liệu. Sau khi thiết kế ban đầu hoàn tất, nhà tư vấn có thể đưa ra quyết định thiết kế về việc lưu trữ dữ liệu nào và dữ liệu nào cần tính toán khi cần. Trong trường hợp này, dữ liệu cơ sở được hiển thị trong Hình 6.1.

Dữ liệu cơ sở trong Hình 6.1 được sắp xếp xung quanh các dự án giống như cách sắp xếp báo cáo, trong đó mỗi dự án có một hàng duy nhất để thể hiện dữ liệu liên quan đến dự án đó. Dữ liệu cơ sở cho thấy một dự án có nhiều nhân viên được giao cho nó. Lưu ý rằng dữ liệu trong Hình 6.1 là **unnormalized data** (dữ liệu không chuẩn hóa), được phản ánh bởi sự tồn tại của một số phần tử dữ liệu đa giá trị (EMP_NUM, EMP_NAME, JOB_CLASS, CHARGE_HOUR, HOURS_BILLED).

unnormalized data (dữ liệu không chuẩn hóa)

Dữ liệu thô ở trạng thái ban đầu; nó có thể chứa dữ liệu dư thừa, dữ liệu đa giá trị và/hoặc các dữ liệu bất thường khác không được tìm thấy trên các quan hệ dữ liệu được chuẩn hóa.

TABLE 6.1

A SAMPLE REPORT LAYOUT

PROJECT NUMBER	PROJECT NAME	EMPLOYEE NUMBER	EMPLOYEE NAME	JOB CLASS	CHARGE/HOUR	HOURS BILLED	TOTAL CHARGE
15	Evergreen	103	June E. Arbough	Elec. Engineer	\$ 84.50	23.8	\$ 2,011.10
		101	John G. News	Database Designer	\$105.00	19.4	\$ 2,037.00
		105	Alice K. Johnson *	Database Designer	\$105.00	35.7	\$ 3,748.50
		106	William Smithfield	Programmer	\$ 35.75	12.6	\$ 450.45
		102	David H. Senior	Systems Analyst	\$ 96.75	23.8	\$ 2,302.65
				Subtotal			\$10,549.70
18	AmberWave	114	Annelise Jones	Applications Designer	\$ 48.10	24.6	\$ 1,183.26
		118	James J. Frommer	General Support	\$ 18.36	45.3	\$ 831.71
		104	Anne K. Ramoras *	Systems Analyst	\$ 96.75	32.4	\$ 3,134.70
		112	Darlene M. Smithson	DSS Analyst	\$ 45.95	44.0	\$ 2,021.80
				Subtotal			\$ 7,171.47
22	Rolling Tide	105	Alice K. Johnson	Database Designer	\$105.00	64.7	\$ 6,793.50
		104	Anne K. Ramoras	Systems Analyst	\$96.75	48.4	\$ 4,682.70
		113	Delbert K. Joenbrood *	Applications Designer	\$48.10	23.6	\$ 1,135.16
		111	Geoff B. Wabash	Clerical Support	\$26.87	22.0	\$ 591.14
		106	William Smithfield	Programmer	\$35.75	12.8	\$ 457.60
				Subtotal			\$13,660.10
25	Starflight	107	Maria D. Alonzo	Programmer	\$ 35.75	24.6	\$ 879.45
		115	Travis B. Bawangi	Systems Analyst	\$ 96.75	45.8	\$ 4,431.15
		101	John G. News *	Database Designer	\$105.00	56.3	\$ 5,911.50
		114	Annelise Jones	Applications Designer	\$ 48.10	33.1	\$ 1,592.11
		108	Ralph B. Washington	Systems Analyst	\$ 96.75	23.6	\$ 2,283.30
		118	James J. Frommer	General Support	\$ 18.36	30.5	\$ 559.98
		112	Darlene M. Smithson	DSS Analyst	\$ 45.95	41.4	\$ 1,902.33
				Subtotal			\$17,559.82
				Total			\$48,941.09

Note: * indicates the project leader.

FIGURE 6.1 BASE DATA FOR A CONSTRUCTION COMPANY REPORT

Table name: RPT_FORMAT

Database name: Ch06_ConstructCo

PROJ_NUM	PROJECT_NAME	EMP_NUMBER	EMP_NAME	JOB_CLASS	CHARGE_HOUR	HOURS_BILLED
15	Evergreen	103,101,105,106,102	June E. Arbough, John G. News, Alice K. Johnson *, William Smithfield, David H. Senior	Elec. Engineer, Database Designer, Database Designer, Programmer, System Analyst	84.5, 105., 105., 35.75, 96.75	23.8, 19.4, 35.7, 12.6, 23.8
18	Amber Wave	114, 118, 104, 112	Annelise Jones, James J. Frommer, Anne K. Ramoras *, Darlene M. Smithson	Applications Designer, General Support, Systems Analyst, DSS Analyst	48.1, 18.36, 96.75, 45.95	25.6, 45.3, 32.4, 45.
22	Rolling Tide	105, 104, 113, 111, 106	Alice K. Johnson, Anne K. Ramoras, Delbert K. Joenbrood *, Geoff B. Wabash, William Smithfield	DB Designer, Systems Analyst, Applications Designer, Clerical Support, Programmer	105., 96.75, 48.1, 26.87, 35.75	65.7, 48.4, 23.6, 22., 12.8
25	Star Light	107, 115, 101, 114, 108, 118, 112	Maria D. Alonzo, Travis B. Bawangi, John G. News *, Annelise Jones, Ralph B. Washington, James J. Frommer, Darlene M. Smithson	Programmer, Systems Analyst, Database Design, Applications Designer, Systems Analyst, General Support, DSS Analyst	35.75, 96.75, 105., 48.1, 96.75, 18.36, 45.95	25.6, 45.8, 56.3, 33.1, 23.6, 30.5, 41.4

Thật không may, cấu trúc dữ liệu được mô tả trong Hình 6.1 không phù hợp với các yêu cầu về bảng quan hệ được thảo luận trong Chương 3 (xem Bảng 3.1) và do đó không phù hợp để xử lý tốt việc cập nhật dữ liệu. Hãy xem xét những thiếu sót sau:

- Cấu trúc dữ liệu gây ra sự không nhất quán về dữ liệu. Ví dụ: giá trị JOB_CLASS “Elect, Engineer” có thể được nhập là “Elect.Eng.” trong một số trường hợp, “El. Eng.” ở những nơi khác, và “EE” ở những nơi khác. Cơ cấu này sẽ cho phép John G. News và Alice K.

Johnson trong dự án Evergreen tính các mức giá khác nhau ngay cả khi họ có cùng phân loại công việc.

- Cấu trúc dữ liệu chứa một số thuộc tính đa giá trị khiến cho việc quản lý dữ liệu trở nên rất khó khăn. Bởi vì tất cả nhân viên làm việc trong một dự án đều ở trong một ô duy nhất nên rất khó để xác định từng nhân viên và để cơ sở dữ liệu trả lời các câu hỏi như “Có bao nhiêu nhân viên đang làm việc trong dự án Starlight?”

- Dữ liệu nhân viên dư thừa trong bảng vì nhân viên có thể làm việc trên nhiều dự án. Việc thêm, cập nhật và xóa dữ liệu có thể sẽ rất cồng kềnh khi sử dụng cấu trúc này. Ví dụ: việc thay đổi phân loại công việc cho Alice K. Johnson sẽ yêu cầu cập nhật ít nhất hai hàng.

Rõ ràng, cấu trúc dữ liệu này tạo ra sự không nhất quán về dữ liệu. Báo cáo có thể mang lại các kết quả khác nhau tùy thuộc vào dữ liệu bất thường nào đã xảy ra. Ví dụ: nếu bạn muốn in một báo cáo để hiển thị tổng số giờ được lập hóa đơn theo phân loại công việc “Nhà thiết kế cơ sở dữ liệu”, thì báo cáo đó sẽ không bao gồm dữ liệu cho các mục nhập dữ liệu “Nhà thiết kế cơ sở dữ liệu” và mục nhập dữ liệu “Thiết kế cơ sở dữ liệu” - giả sử thậm chí có thể phân tích cú pháp thông qua nhiều giá trị trong mỗi ô của cột phân loại công việc để phân biệt giữa các phân loại công việc khác nhau. Những bất thường trong báo cáo như vậy gây ra vô số vấn đề cho người quản lý - và không thể khắc phục được bằng lập trình ứng dụng.

Các vấn đề về tính toàn vẹn dữ liệu, sự dư thừa dữ liệu và tính không nhất quán của dữ liệu phải được giải quyết trong quá trình thiết kế cơ sở dữ liệu. Phần tiếp theo sẽ hướng dẫn bạn quy trình chuẩn hóa được sử dụng để giảm thiểu sự dư thừa và loại bỏ các dữ liệu bất thường.

6-3. Quá trình chuẩn hóa

Trong phần này, bạn tìm hiểu cách sử dụng chuẩn hóa để tạo ra một tập hợp các quan hệ (bảng) chuẩn hóa sẽ được sử dụng để tạo ra thông tin cần thiết. Mục tiêu của việc chuẩn hóa là đảm bảo rằng mỗi bảng tuân theo khái niệm về các mối quan hệ được định dạng đúng—nói cách khác, các bảng có các đặc điểm sau:

- Mỗi mối quan hệ (bảng) đại diện cho một chủ đề duy nhất. Ví dụ: bảng COURSE sẽ chỉ chứa dữ liệu liên quan trực tiếp đến các khóa học. Tương tự, bảng STUDENT sẽ chỉ chứa dữ liệu về sinh viên.
- Mỗi giao điểm hàng/cột chỉ chứa một giá trị chứ không chứa một nhóm giá trị.
- Không có mục dữ liệu nào được lưu trữ *một cách không cần thiết* trong nhiều bảng (các bảng có độ dư thừa được kiểm soát tối thiểu). Lý do cho yêu cầu này là để đảm bảo dữ liệu chỉ được cập nhật ở một nơi.
- Tất cả các thuộc tính không phải số nguyên tố trong một quan hệ (bảng) đều phụ thuộc vào khóa chính - toàn bộ khóa chính và không có gì ngoài khóa chính. Lý do cho yêu cầu này là để đảm bảo rằng dữ liệu có thể được nhận dạng duy nhất bằng giá trị khóa chính.
- Mỗi mối quan hệ (bảng) không có các bất thường về chèn, cập nhật hoặc xóa, điều này đảm bảo tính toàn vẹn và nhất quán của dữ liệu.

Để hoàn thành các mục tiêu này, quá trình chuẩn hóa sẽ đưa bạn qua các bước dẫn đến các dạng chuẩn hóa cao hơn liên tiếp. Các dạng chuẩn tắc phổ biến nhất và các đặc tính cơ

bản của chúng được liệt kê trong Bảng 6.2. Chi tiết về các biểu mẫu thông thường này được cung cấp trong các phần được chỉ định.

TABLE 6.2		
NORMAL FORMS		
NORMAL FORM	CHARACTERISTIC	SECTION
First normal form (1NF)	Table format, no repeating groups, and PK identified	6-3a
Second normal form (2NF)	1NF and no partial dependencies	6-3b
Third normal form (3NF)	2NF and no transitive dependencies	6-3c
Boyce-Codd normal form (BCNF)	Every determinant is a candidate key (special case of 3NF)	6-6a
Fourth normal form (4NF)	3NF and no independent multivalued dependencies	6-6b

Khái niệm về khóa là trọng tâm của cuộc thảo luận về chuẩn hóa. Hãy nhớ lại ở Chương 3 rằng khóa ứng viên là một siêu khóa tối thiểu (không thể rút gọn). Khóa chính là khóa dự tuyển được chọn làm phương tiện chính được sử dụng để xác định các hàng trong bảng. Mặc dù việc chuẩn hóa thường được trình bày từ góc độ của các khóa ứng viên, nhưng cuộc thảo luận ban đầu này giả định rằng vì mục đích đơn giản, mỗi bảng chỉ có một khóa ứng viên; do đó, khóa ứng viên đó là khóa chính.

Từ quan điểm của người lập mô hình dữ liệu, mục tiêu của việc chuẩn hóa là đảm bảo rằng tất cả các bảng ít nhất ở dạng 3NF. Thậm chí còn tồn tại những dạng chuẩn ở cấp độ cao hơn. Tuy nhiên, các dạng chuẩn như dạng chuẩn thứ năm (5NF) và dạng chuẩn khóa miền (DKNF) không có khả năng gặp trong môi trường nghiệp vụ và chủ yếu được quan tâm về mặt lý thuyết. Các dạng chuẩn cao hơn như vậy thường làm tăng các phép nối, điều này làm chậm hiệu suất mà không thêm bất kỳ giá trị nào trong việc loại bỏ sự dư thừa dữ liệu. Một số ứng dụng rất chuyên biệt, chẳng hạn như nghiên cứu thống kê, có thể yêu cầu chuẩn hóa ngoài 4NF, nhưng những ứng dụng đó nằm ngoài phạm vi của hầu hết các hoạt động nghiệp vụ. Bởi vì cuốn sách này tập trung vào các ứng dụng thực tế của các kỹ thuật cơ sở dữ liệu nên không đề cập đến các dạng chuẩn cấp cao hơn.

Trước khi phác thảo quá trình chuẩn hóa, bạn nên xem lại các khái niệm xác định và sự phụ thuộc hàm đã được đề cập chi tiết trong Chương 3. Bảng 6.3 tóm tắt các khái niệm chính.

Điều quan trọng là phải hiểu các khái niệm này vì chúng được sử dụng để suy ra tập hợp các phụ thuộc hàm cho một quan hệ nhất định. Quá trình chuẩn hóa thực hiện từng mối quan hệ tại một thời điểm, xác định các phụ thuộc vào mối quan hệ đó và chuẩn hóa mối quan hệ. Như bạn sẽ thấy trong các phần sau, quá trình chuẩn hóa bắt đầu bằng cách xác định các mối quan hệ phụ thuộc của một mối quan hệ nhất định và dần dần chia nhỏ mối quan hệ (bảng) thành một tập hợp các mối quan hệ (bảng) mới dựa trên các mối quan hệ đã xác định.

TABLE 6.3	
FUNCTIONAL DEPENDENCE CONCEPTS	
CONCEPT	DEFINITION
Functional dependence	The attribute <i>B</i> is fully functionally dependent on the attribute <i>A</i> if each value of <i>A</i> determines one and only one value of <i>B</i> . Example: $PROJ_NUM \rightarrow PROJ_NAME$ (read as <i>PROJ_NUM functionally determines PROJ_NAME</i>) In this case, the attribute <i>PROJ_NUM</i> is known as the determinant attribute, and the attribute <i>PROJ_NAME</i> is known as the dependent attribute.
Functional dependence (generalized definition)	Attribute <i>A</i> determines attribute <i>B</i> (that is, <i>B</i> is functionally dependent on <i>A</i>) if all (generalized definition) of the rows in the table that agree in value for attribute <i>A</i> also agree in value for attribute <i>B</i> .
Fully functional dependence (composite key)	If attribute <i>B</i> is functionally dependent on a composite key <i>A</i> but not on any subset of that composite key, the attribute <i>B</i> is fully functionally dependent on <i>A</i> .

Hai loại phụ thuộc hàm được đặc biệt quan tâm trong quá trình chuẩn hóa là phụ thuộc từng phần và phụ thuộc bắc cầu. **A partial dependency** (phụ thuộc một phần)

Sự phụ thuộc một phần tồn tại khi có sự phụ thuộc hàm trong đó định thức chỉ là một phần của khóa chính (hãy nhớ giả định rằng, đối với cuộc thảo luận này, chỉ có một khóa ứng cử viên). Ví dụ: nếu $(A, B) \rightarrow (C, D)$, $B \rightarrow C$ và (A, B) là khóa chính thì sự phụ thuộc hàm $B \rightarrow C$ là phụ thuộc một phần vì chỉ một phần của khóa chính (*B*) cần thiết để xác định giá trị của *C*. Các phụ thuộc một phần có xu hướng đơn giản và dễ xác định.

A transitive dependency (sự phụ thuộc bắc cầu) tồn tại khi có các phụ thuộc hàm như $X \rightarrow Y$, $Y \rightarrow Z$ và *X* là khóa chính. Trong trường hợp đó, phụ thuộc $X \rightarrow Z$ là phụ thuộc bắc cầu vì *X* xác định giá trị của *Z* thông qua *Y*. Không giống như phụ thuộc một phần, phụ thuộc bắc cầu khó xác định hơn giữa một tập hợp dữ liệu. May mắn thay, có một cách hiệu quả để xác định sự phụ thuộc bắc cầu: chúng chỉ xảy ra khi sự phụ thuộc chức năng tồn tại giữa các thuộc tính không nguyên tố. Trong ví dụ trước, sự phụ thuộc bắc cầu thực tế là $X \rightarrow Z$. Tuy nhiên, sự phụ thuộc $Y \rightarrow Z$ báo hiệu rằng có tồn tại một phụ thuộc bắc cầu. Do đó, trong suốt cuộc thảo luận về quá trình chuẩn hóa, sự tồn tại của sự phụ thuộc hàm giữa các thuộc tính không nguyên tố sẽ được coi là dấu hiệu của sự phụ thuộc bắc cầu. Để giải quyết các vấn đề liên quan đến sự phụ thuộc bắc cầu, các thay đổi đối với cấu trúc bảng

partial dependency (phụ thuộc một phần)

Điều kiện trong đó thuộc tính chỉ phụ thuộc vào một phần (tập hợp con) của khóa chính.

transitive dependency (sự phụ thuộc bắc cầu)

Điều kiện trong đó một thuộc tính phụ thuộc vào một thuộc tính khác không phải là một phần của khóa chính.

repeating group (nhóm lặp lại)

Trong một mối quan hệ, một đặc tính mô tả một nhóm gồm nhiều mục cùng loại hoặc nhiều loại cho một lần xuất hiện thuộc tính khóa duy nhất. Ví dụ: một chiếc ô tô có thể có nhiều màu sắc cho phần trên, phần bên trong, phần dưới, phần trang trí, v.v.

được thực hiện dựa trên sự phụ thuộc chức năng báo hiệu sự tồn tại của sự phụ thuộc bắc cầu. Do đó, để đơn giản hóa việc mô tả chuẩn hóa, từ thời điểm này trở đi, sự phụ thuộc tín hiệu sẽ được gọi là *sự phụ thuộc bắc cầu*.

6-3a. Chuyển đổi sang dạng chuẩn thứ nhất (1NF)

Bởi vì mô hình quan hệ xem dữ liệu như một phần của bảng hoặc một tập hợp các bảng trong đó tất cả các giá trị khóa phải được xác định, dữ liệu được mô tả trong Hình 6.1 có thể không được lưu trữ tốt nhất như được hiển thị. Lưu ý rằng Hình 6.1 chứa cái được gọi là nhóm lặp lại. A **repeating group** (nhóm lặp lại) lấy tên của nó từ thực tế là một nhóm gồm nhiều mục cùng loại hoặc nhiều loại có thể tồn tại cho bất kỳ lần xuất hiện thuộc tính khóa đơn lẻ nào. Trong Hình 6.1, lưu ý rằng mỗi lần xuất hiện mã số dự án (PROJ_NUM) có thể tham chiếu một nhóm dữ liệu liên quan trong mã số nhân viên, tên nhân viên, phân loại công việc và cột giờ tính phí. Ví dụ: dự án Evergreen (PROJ_NUM = 15) chứa năm giá trị cho mỗi thuộc tính đó tại thời điểm này.

Việc chuẩn hóa cấu trúc bảng sẽ làm giảm sự dư thừa dữ liệu. Nếu các nhóm lặp lại tồn tại, chúng phải được loại bỏ bằng cách đảm bảo rằng mỗi hàng xác định một thể hiện thực thể đơn lẻ và mỗi giao điểm hàng-cột chỉ có một giá trị duy nhất. Ngoài ra, các phụ thuộc phải được xác định để chẩn đoán dạng chuẩn. Việc xác định dạng chuẩn cho phép bạn biết mình đang ở đâu trong quá trình chuẩn hóa. Quá trình chuẩn hóa bắt đầu bằng quy trình ba bước đơn giản.

Lưu ý:

Mục đích ở đây là để minh họa quá trình chuẩn hóa và các dạng chuẩn hóa khác nhau. Chương 4 đã trình bày một giải pháp mạnh mẽ hơn để xử lý các thuộc tính đa giá trị bằng cách tạo một thực thể mới. Giải pháp mạnh mẽ hơn này thực sự hợp nhất nhiều bước từ quy trình chuẩn hóa có liên quan cụ thể đến các thuộc tính đa giá trị. Tuy nhiên, giải pháp đó không nhất thiết có thể áp dụng cho tất cả các vấn đề nhóm lặp lại, vì vậy điều quan trọng là phải hiểu mục tiêu chuẩn hóa của dạng chuẩn hóa đầu tiên.

Bước 1: Loại bỏ các nhóm lặp lại

Bắt đầu bằng cách trình bày dữ liệu ở định dạng bảng, trong đó mỗi ô có một giá trị duy nhất và không có nhóm lặp lại. Để loại bỏ các nhóm lặp lại, hãy thay đổi bảng từ tiêu điểm dự án thành tiêu điểm phân công. Điều này sẽ tạo các hàng riêng biệt cho mỗi nhân viên được giao cho từng dự án, chuyển đổi các thuộc tính đa giá trị thành thuộc tính một giá trị. Sự thay đổi này chuyển đổi bảng trong Hình 6.1 thành 1NF như trong Hình 6.2

FIGURE 6.2 A TABLE IN FIRST NORMAL FORM

Table name: DATA_ORG_1NF

Database name: Ch06_ConstructCo

PROJ_NUM	PROJ_NAME	EMP_NUM	EMP_NAME	JOB_CLASS	CHG_HOUR	HOURS
15	Evergreen	103	June E. Arbough	Elect. Engineer	84.50	23.8
15	Evergreen	101	John G. News	Database Designer	105.00	19.4
15	Evergreen	105	Alice K. Johnson *	Database Designer	105.00	35.7
15	Evergreen	106	William Smithfield	Programmer	35.75	12.6
15	Evergreen	102	David H. Senior	Systems Analyst	96.75	23.8
18	Amber Wave	114	Annelise Jones	Applications Designer	48.10	24.6
18	Amber Wave	118	James J. Frommer	General Support	18.36	45.3
18	Amber Wave	104	Anne K. Ramoras *	Systems Analyst	96.75	32.4
18	Amber Wave	112	Darlene M. Smithson	DSS Analyst	45.95	44.0
22	Rolling Tide	105	Alice K. Johnson	Database Designer	105.00	64.7
22	Rolling Tide	104	Anne K. Ramoras	Systems Analyst	96.75	48.4
22	Rolling Tide	113	Delbert K. Joenbrood *	Applications Designer	48.10	23.6
22	Rolling Tide	111	Geoff B. Wabash	Clerical Support	26.87	22.0
22	Rolling Tide	106	William Smithfield	Programmer	35.75	12.8
25	Starflight	107	Maria D. Alonzo	Programmer	35.75	24.6
25	Starflight	115	Travis B. Bawangi	Systems Analyst	96.75	45.8
25	Starflight	101	John G. News *	Database Designer	105.00	56.3
25	Starflight	114	Annelise Jones	Applications Designer	48.10	33.1
25	Starflight	108	Ralph B. Washington	Systems Analyst	96.75	23.6
25	Starflight	118	James J. Frommer	General Support	18.36	30.5
25	Starflight	112	Darlene M. Smithson	DSS Analyst	45.95	41.4

Bước 2: Xác định khóa chính

Bố cục trong Hình 6.2 không chỉ thể hiện một sự thay đổi bề ngoài đơn thuần. Ngay cả một người quan sát bình thường cũng sẽ lưu ý rằng PROJ_NUM không phải là khóa chính đầy đủ vì số dự án không xác định duy nhất từng hàng. Ví dụ: giá trị PROJ_NUM = 15 có thể xác định bất kỳ một trong năm hàng chứa nhân viên làm việc trong dự án Evergreen. Để duy trì khóa chính thích hợp sẽ xác định *duy nhất* bất kỳ giá trị thuộc tính nào, khóa mới phải bao gồm sự *kết hợp* của PROJ_NUM và EMP_NUM. Ví dụ: sử dụng dữ liệu hiển thị trong Hình 6.2, nếu bạn biết PROJ_NUM = 15 và EMP_NUM = 103 thì các mục nhập cho các thuộc tính PROJ_NAME, EMP_NAME, JOB_CLASS, CHG_HOUR và HOURS phải là Evergreen, June E. Arbough, Elect. Engineer, \$84.50, và 23.8

Bước 3: Xác định tất cả các phụ thuộc

Việc xác định PK ở Bước 2 có nghĩa là bạn đã xác định phụ thuộc sau:

PROJ_NUM, EMP_NUM → PROJ_NAME, EMP_NAME, JOB_CLASS, CHG_HOUR, HOURS

Đó là các giá trị PROJ_NAME, EMP_NAME, JOB_CLASS, CHG_HOUR và HOURS tất cả đều phụ thuộc vào - chúng được xác định bởi - sự kết hợp của PROJ_NUM và EMP_NUM.

Việc đạt được 1NF là không đủ để giải quyết tất cả các điểm bất thường tồn tại trong cấu trúc ban đầu. 1NF đã xử lý các nhóm lặp lại và đảm bảo rằng bảng của chúng tôi tuân thủ các yêu cầu đối với bảng quan hệ, như được mô tả trong Chương 3. Tuy nhiên, vẫn còn những điểm bất thường. Ví dụ: mỗi khi một nhân viên khác được phân công vào một dự án, một số mục nhập dữ liệu (chẳng hạn như PROJ_NAME, EMP_NAME và

CHG_HOUR) được lặp lại một cách không cần thiết. Hãy tưởng tượng công việc nhập dữ liệu khi phải thực hiện 200 hoặc 300 mục trong bảng! Lý tưởng nhất là việc nhập mã số nhân viên phải đủ để xác định DarleneM. Smithson, bản mô tả công việc và mức lương theo giờ của cô ấy. Vì chỉ có một người được xác định bằng số 112 nên không cần phải nhập các đặc điểm của người đó (tên, phân loại công việc, v.v.) mỗi khi thực hiện hoặc cập nhật nhiệm vụ.

Những điểm bất thường vẫn tồn tại vì có những phụ thuộc bổ sung ngoài sự phụ thuộc khóa chính. Ví dụ: số dự án xác định tên dự án. Nói cách khác, tên dự án phụ thuộc vào số dự án. Bạn có thể viết phần phụ thuộc đó là:

PROJ_NUM → PROJ_NAME

Ngoài ra, nếu bạn biết mã số nhân viên, bạn cũng biết tên nhân viên đó, phân loại công việc và mức phí mỗi giờ. Do đó, bạn có thể xác định phần phụ thuộc được hiển thị tiếp theo:

EMP_NUM → EMP_NAME, JOB_CLASS, CHG_HOUR

Nói một cách đơn giản hơn, một nhân viên có các thuộc tính sau: số, tên, phân loại công việc và mức phí mỗi giờ.

Lưu ý:

Một số phụ thuộc rõ ràng hơn những phụ thuộc khác. Ví dụ: quy tắc nghiệp vụ “Mỗi phân loại công việc có một mức phí theo giờ cụ thể” ngụ ý rằng mức phí theo giờ phụ thuộc vào phân loại công việc. Tuy nhiên, hãy nhớ rằng các cuộc thảo luận trong chương này dựa trên quan điểm của quá trình chuẩn hóa và dùng để chỉ ra cách chuẩn hóa cũng có thể giúp xác thực các quy tắc nghiệp vụ.

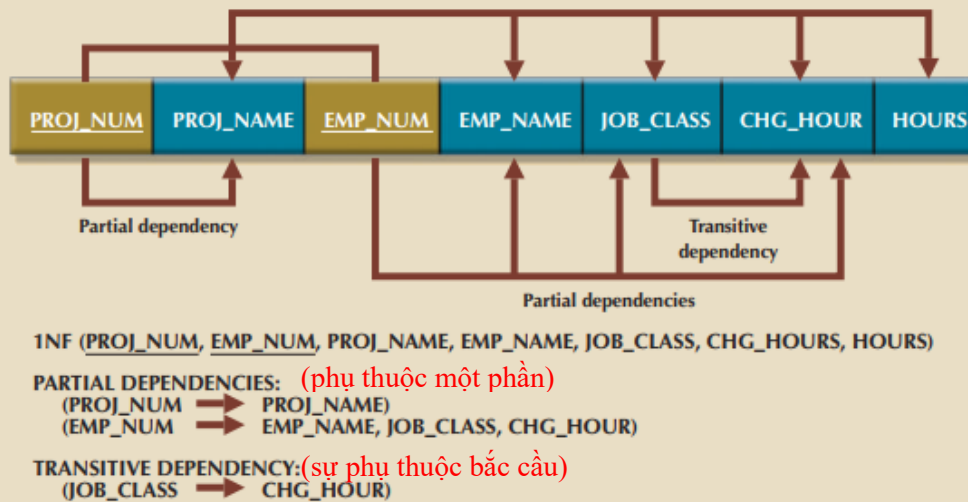
Bằng cách nghiên cứu sâu hơn về dữ liệu trong Hình 6.2, bạn có thể thấy rằng biết phân loại công việc có nghĩa là biết mức phí mỗi giờ cho phân loại công việc đó. (Lưu ý rằng tất cả các vị trí Nhà phân tích hệ thống hoặc Lập trình viên đều có cùng mức phí mỗi giờ bất kể dự án hay nhân viên.) Nói cách khác, mức phí mỗi giờ phụ thuộc vào phân loại công việc chứ không phải nhân viên. Do đó, bạn có thể xác định một phụ thuộc cuối cùng:

JOB_CLASS → CHG_HOUR

Tuy nhiên, sự phụ thuộc này tồn tại giữa hai thuộc tính không phải số nguyên tố; do đó, đó là tín hiệu cho thấy sự phụ thuộc bắc cầu tồn tại và chúng ta sẽ gọi nó là sự phụ thuộc bắc cầu. Các mối phụ thuộc mà bạn vừa kiểm tra cũng có thể được mô tả bằng sơ đồ trong Hình 6.3. Vì sơ đồ như vậy mô tả tất cả các phần phụ thuộc được tìm thấy trong một cấu trúc bảng nhất định nên nó được gọi là **dependency diagram** (sơ đồ phụ thuộc). Sơ đồ phụ thuộc rất hữu ích trong việc có được cái nhìn toàn cảnh về tất cả các mối quan hệ giữa các thuộc tính của bảng và việc sử dụng chúng khiến bạn ít có khả năng bỏ qua sự phụ thuộc quan trọng.

Dependency diagram
(Biểu đồ phụ thuộc)
Một biểu đồ biểu thị tất cả các phụ thuộc dữ liệu (khóa chính, phụ thuộc một phần hoặc chuyển tiếp) trong một bảng.

FIGURE 6.3 FIRST NORMAL FORM (1NF) DEPENDENCY DIAGRAM



Khi bạn xem xét Hình 6.3, hãy lưu ý các đặc điểm sau của sơ đồ phụ thuộc:

1. Các thuộc tính khóa chính được in đậm, gạch chân và có màu khác.
2. Các mũi tên phía trên các thuộc tính chỉ ra tất cả các phụ thuộc mong muốn—nghĩa là các phụ thuộc dựa trên khóa chính. Trong trường hợp này, hãy lưu ý rằng các thuộc tính của thực thể phụ thuộc vào sự *kết hợp* của PROJ_NUM và EMP_NUM.
3. Các mũi tên bên dưới sơ đồ phụ thuộc biểu thị các phụ thuộc ít được mong muốn hơn. Có hai loại phụ thuộc như vậy tồn tại:

a. *Phụ thuộc một phần*. Bạn chỉ cần biết PROJ_NUM để xác định PROJ_NAME; nghĩa là PROJ_NAME chỉ phụ thuộc vào một phần của khóa chính. Ngoài ra, bạn chỉ cần biết EMP_NUM để tìm EMP_NAME, JOB_CLASS và CHG_HOUR. Sự phụ thuộc chỉ dựa trên một phần của khóa chính tổng hợp được coi là sự phụ thuộc một phần.

b. *Sự phụ thuộc bắc cầu*. Lưu ý rằng CHG_HOUR phụ thuộc vào JOB_CLASS. Bởi vì cả CHG_HOUR và JOB_CLASS đều không phải là thuộc tính chính - nghĩa là cả hai thuộc tính đều không ít nhất là một phần của khóa - điều kiện biểu thị sự phụ thuộc bắc cầu. Nói cách khác, sự phụ thuộc bắc cầu tồn tại khi sự phụ thuộc hàm chỉ tồn tại giữ

a các thuộc tính không nguyên tố. Sự phụ thuộc bắc cầu mang lại sự bất thường về dữ liệu.

first normal form

(dạng chuẩn đầu tiên)
(1NF)

Giai đoạn đầu tiên trong quá trình chuẩn hóa. Nó mô tả mỗi quan hệ được mô tả dưới dạng bảng, không có nhóm lặp lại và khóa chính được xác định. Tất cả các thuộc tính không khóa trong quan hệ đều phụ thuộc vào khóa chính

Hình 6.3 bao gồm lược đồ quan hệ cho bảng ở dạng 1NF và ký hiệu văn bản cho từng phụ thuộc được xác định.

Lưu ý:

Thuật ngữ dạng chuẩn đầu tiên (1NF) mô tả định dạng dạng bảng phù hợp với định nghĩa của bảng quan hệ trong đó:

- Tất cả các thuộc tính chính được xác định.
- Không có nhóm lặp lại trong bảng. Nói cách khác, mỗi giao điểm hàng/cột chứa một và chỉ một giá trị, không phải một tập hợp các giá trị.
- Tất cả các thuộc tính đều phụ thuộc vào khóa chính.

Tất cả các bảng quan hệ đều đáp ứng các yêu cầu 1NF. Mặc dù dữ liệu 1NF trong Hình 6.2 là một cải tiến so với dữ liệu không chuẩn hóa trong Hình 6.1, nhưng nó vẫn có những vấn đề không mong muốn. Ví dụ: cấu trúc bảng 1NF được hiển thị trong Hình 6.2 và được biểu thị bằng sơ đồ phụ thuộc trong Hình 6.3 chứa các phụ thuộc một phần và các phụ thuộc bắc cầu gây ra các bất thường về dữ liệu giống như chúng ta đã khám phá trước đó.

Mặc dù các phụ thuộc một phần đôi khi được sử dụng vì lý do hiệu suất nhưng chúng nên được sử dụng một cách thận trọng bởi vì một bảng chứa các phần phụ thuộc một phần vẫn có thể bị dư thừa dữ liệu và do đó có nhiều điểm bất thường khác nhau. Ví dụ của chúng tôi vẫn có những điểm bất thường sau:

a. *Cập nhật những bất thường* . Sửa đổi JOB_CLASS cho nhân viên Annelise Jones yêu cầu

cập nhật nhiều mục; nếu không, nó sẽ tạo ra sự không nhất quán về dữ liệu.

b. *Sự chèn bất thường* . Việc thêm nhân viên mới yêu cầu nhân viên đó phải được phân công vào một dự án và do đó phải nhập thông tin dự án trùng lặp. Nếu nhân viên chưa được phân công vào một dự án, một dự án ảo phải được tạo để hoàn thành mục nhập dữ liệu nhân viên.

c. *Xóa bất thường* . Giả sử chỉ có một nhân viên được liên kết với một dự án nhất định. Nếu nhân viên đó bị xóa, thông tin dự án cũng sẽ bị xóa.

Sự dư thừa dữ liệu xảy ra do mỗi mục nhập hàng đều yêu cầu sao chép dữ liệu. Việc sao chép nỗ lực như vậy rất kém hiệu quả và nó giúp tạo ra sự bất thường về dữ liệu; không có gì ngăn cản người dùng nhập các phiên bản hơi khác nhau của tên nhân viên, vị trí hoặc lương theo giờ. Ví dụ: tên nhân viên cho EMP_NUM = 102 có thể được nhập là *Dave Senior* hoặc *D. Senior* . Tên dự án cũng có thể được nhập chính xác là *Evergreen* hoặc viết sai chính tả là *Evergeen* . Những dữ liệu bất thường như vậy vi phạm các quy tắc nhất quán và toàn vẹn của cơ sở dữ liệu quan hệ .

6-3b. Chuyển đổi sang dạng chuẩn thứ hai (2NF)

Chuyển đổi sang 2NF chỉ xảy ra khi 1NF có khóa chính tổng hợp. Nếu 1NF có khóa chính thuộc tính đơn thì bảng sẽ tự động ở dạng 2NF. Việc chuyển đổi 1NF-to-2NF rất đơn giản. Bắt đầu với định dạng 1NF được hiển thị trong Hình 6.3, bạn thực hiện các bước sau:

Bước 1: Tạo bảng mới để loại bỏ sự phụ thuộc một phần

Đối với mỗi thành phần của khóa chính đóng vai trò là yếu tố quyết định trong phụ thuộc một phần, hãy tạo một bảng mới với một bản sao của thành phần đó làm khóa chính. Mặc dù các thành phần này được đặt trong các bảng mới, nhưng điều quan trọng là chúng cũng vẫn còn trong bảng gốc. Các định thức phải vẫn còn trong bảng gốc vì chúng sẽ là khóa ngoại cho các mối quan hệ cần thiết để liên kết các bảng mới này với bảng gốc. Để xây dựng sơ đồ phụ thuộc đã sửa đổi, hãy viết từng thành phần chính trên một dòng riêng biệt và sau đó viết khóa gốc (tổng hợp) trên dòng cuối cùng. Ví dụ:

PROJ_NUM
EMP_NUM
PROJ_NUM EMP_NUM

Mỗi thành phần sẽ trở thành khóa trong một bảng mới. Nói cách khác, bảng gốc hiện được chia thành ba bảng (PROJECT, EMPLOYEE, và ASSIGNMENT).

Bước 2: Gán lại các thuộc tính phụ thuộc tương ứng

Sử dụng Hình 6.3 để xác định các thuộc tính phụ thuộc vào các phụ thuộc một phần. Sự phụ thuộc của các thành phần khóa ban đầu được tìm thấy bằng cách kiểm tra các mũi tên bên dưới sơ đồ phụ thuộc được hiển thị trong Hình 6.3. Các thuộc tính phụ thuộc vào một phần phụ thuộc sẽ bị xóa khỏi bảng gốc và được đặt vào bảng mới với định thức của phần phụ thuộc. Bất kỳ thuộc tính nào không phụ thuộc vào sự phụ thuộc một phần sẽ vẫn còn trong bảng gốc. Nói cách khác, ba bảng là kết quả của việc chuyển đổi sang 2NF được đặt tên thích hợp (PROJECT, EMPLOYEE, và ASSIGNMENT) và được mô tả bằng các lược đồ quan hệ sau:

PROJECT (PROJ_NUM , PROJ_NAME)

EMPLOYEE (EMP_NUM , EMP_NAME , JOB_CLASS , CHG_HOUR)

ASSIGNMENT (PROJ_NUM, EMP_NUM , ASSIGN_HOURS)

Bởi vì số giờ mỗi nhân viên dành cho mỗi dự án phụ thuộc vào cả PROJ_NUM và EMP_NUM trong bảng ASSIGNMENT, nên bạn để số giờ đó trong bảng ASSIGNMENT là ASSIGN_HOURS. Lưu ý rằng bảng ASSIGNMENT chứa khóa chính tổng hợp bao gồm các thuộc tính PROJ_NUM và EMP_NUM. Lưu ý rằng bằng cách để các định thức trong bảng gốc cũng như biến chúng thành khóa chính của các

bảng mới, mỗi quan hệ khóa chính/khóa ngoại đã được tạo. Ví dụ: trong bảng EMPLOYEE, EMP_NUM là khóa chính. Trong bảng ASSIGNMENT, EMP_NUM là một phần của khóa chính tổng hợp (PROJ_NUM, EMP_NUM) và là khóa ngoại liên kết bảng EMPLOYEE với bảng ASSIGNMENT. Kết quả của Bước 1 và 2 được hiển thị trong Hình 6.4. Tại thời điểm này, hầu hết các điểm bất thường được thảo luận trước đó đã được loại bỏ. Ví dụ: nếu bây giờ bạn muốn thêm, thay đổi hoặc xóa một bản ghi PROJECT, bạn chỉ cần đi tới bảng PROJECT và thực hiện thay đổi chỉ một hàng. Bởi vì một phần phụ thuộc chỉ có thể tồn tại khi khóa chính của bảng bao gồm nhiều thuộc tính, một bảng có khóa chính chỉ bao gồm một thuộc tính duy nhất sẽ tự động ở dạng 2NF khi nó ở dạng 1NF.

second normal form
(dạng chuẩn thứ hai)
(2NF)
Giai đoạn thứ hai trong quá trình bình thường hóa, trong đó mỗi quan hệ đã đạt đến 1NF và không có sự phụ thuộc một phần (phụ thuộc chỉ vào một phần của khóa chính).

Lưu ý:

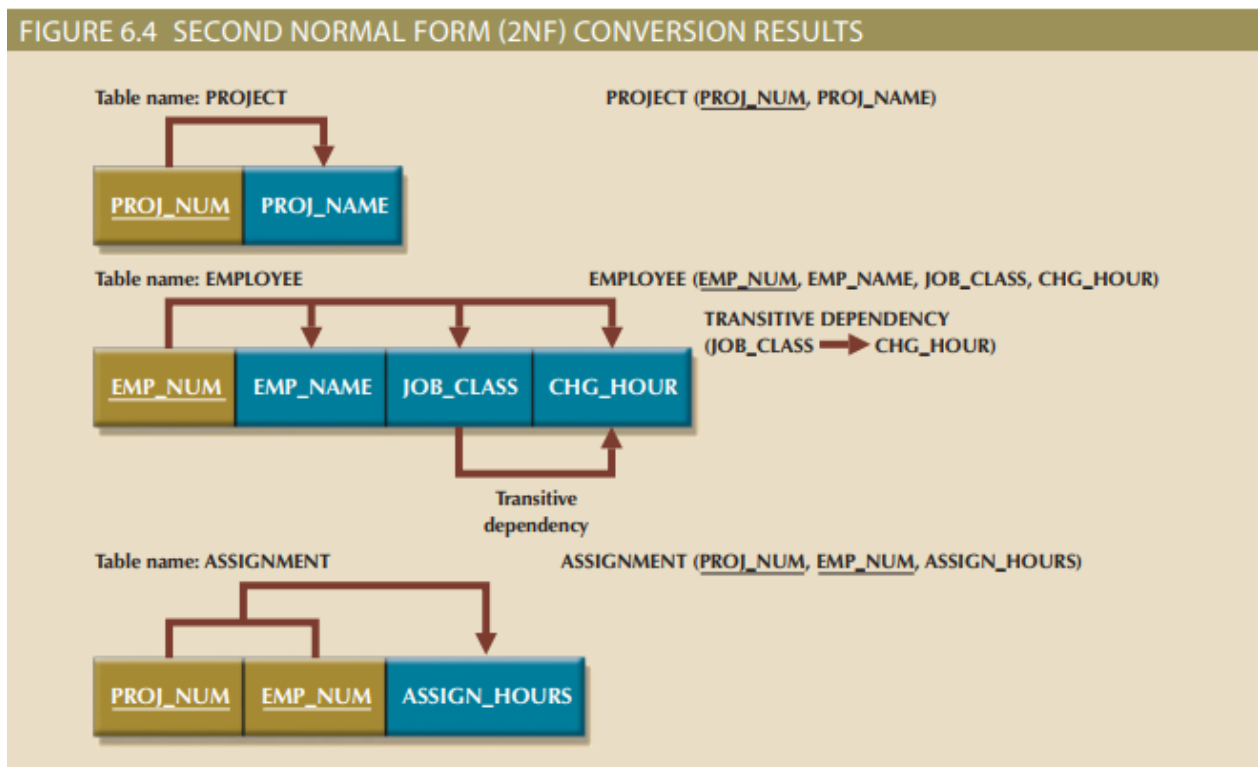
Một bảng ở dạng chuẩn thứ hai (2NF) khi:

- Nó ở dạng 1NF.

và

- Nó không bao gồm sự phụ thuộc một phần; nghĩa là, không có thuộc tính nào chỉ phụ thuộc vào một phần của khóa chính.

Một bảng ở 2NF vẫn có thể thể hiện sự phụ thuộc bắc cầu. Nghĩa là, khóa chính có thể dựa vào một hoặc nhiều thuộc tính không chính để xác định một cách chức năng các thuộc tính không chính khác, được biểu thị bằng sự phụ thuộc chức năng giữa các thuộc tính không chính



Hình 6.4 vẫn cho thấy sự phụ thuộc bắc cầu, có thể tạo ra sự bất thường. Ví dụ: nếu mức phí theo giờ thay đổi đối với phân loại công việc do nhiều nhân viên nắm giữ thì thay đổi đó phải được thực hiện cho *từng* nhân viên đó. Nếu bạn quên cập nhật một số hồ sơ nhân viên bị ảnh hưởng bởi sự thay đổi về phí theo giờ, những nhân viên khác nhau có cùng mô tả công việc sẽ tạo ra các khoản phí theo giờ khác nhau.

6-3c. Chuyển đổi sang dạng chuẩn thứ ba (3NF)

Các dữ liệu dư thừa được tạo bởi tổ chức cơ sở dữ liệu được hiển thị trong Hình 6.4 có thể dễ dàng loại bỏ bằng cách hoàn thành hai bước sau:

Bước 1: Tạo các bảng mới để loại bỏ các phụ thuộc bắc cầu

Đối với mỗi phụ thuộc bắc cầu, hãy viết một bản sao của định thức của nó làm khóa chính cho một bảng mới. A **determinant** (định thức) là bất kỳ thuộc tính nào có giá trị xác định các giá trị khác trong một hàng. Nếu bạn có ba phụ thuộc bắc cầu khác nhau, bạn sẽ có ba yếu tố quyết định khác nhau. Khi chuyển đổi sang 2NF, điều quan trọng là yếu tố quyết

định đó vẫn còn trong bảng gốc để dùng làm khóa ngoại. Hình 6.4 chỉ hiển thị một bảng chứa phụ thuộc bậc cao. Do đó, hãy viết định thức cho sự phụ thuộc bậc cao này là: JOB_CLASS

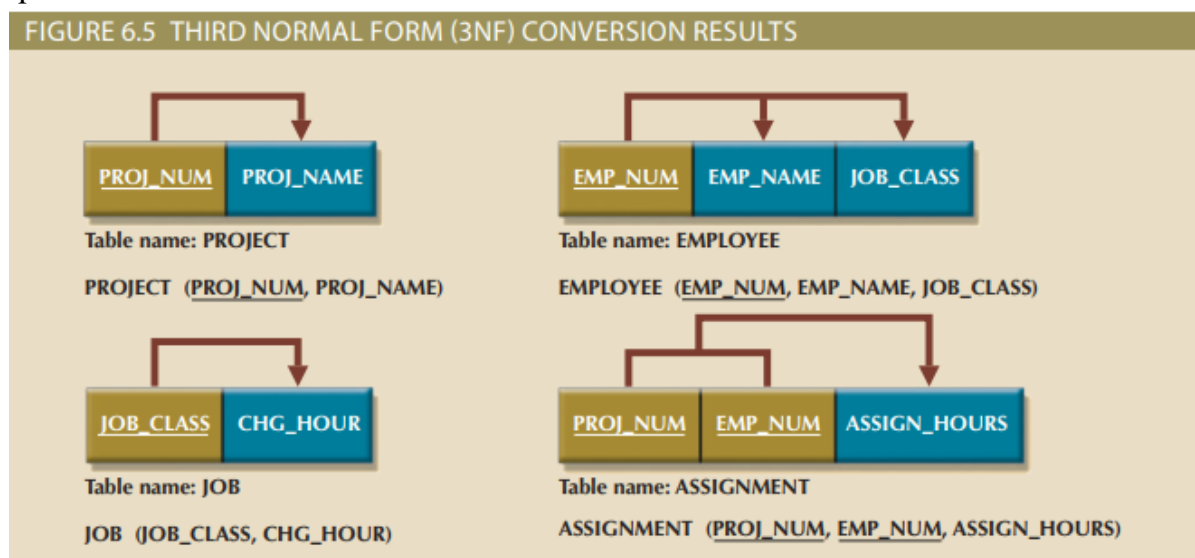
Determinant (định thức)
Bất kỳ thuộc tính nào trong một hàng cụ thể có giá trị xác định trực tiếp các giá trị khác trong hàng đó

Bước 2: Gán lại các thuộc tính phụ thuộc tương ứng

Sử dụng Hình 6.4, xác định các thuộc tính phụ thuộc vào từng định thức được xác định trong Bước 1. Đặt các thuộc tính phụ thuộc vào các bảng mới cùng với các định thức của chúng và loại bỏ chúng từ các bảng ban đầu của họ. Trong ví dụ này, loại bỏ CHG_HOUR khỏi bảng EMPLOYEE được hiển thị trong Hình 6.4 để giữ lại định nghĩa phụ thuộc bảng EMPLOYEE là:

EMP_NUM → EMP_NAME, JOB_CLASS

Vẽ sơ đồ phụ thuộc mới để hiển thị tất cả các bảng bạn đã xác định ở Bước 1 và 2. Đặt tên cho bảng để phản ánh nội dung và chức năng của bảng. Trong trường hợp này, JOB có vẻ phù hợp. Kiểm tra tất cả các bảng để đảm bảo rằng mỗi bảng có định thức và bảng đáng chú ý chứa các phụ thuộc không phù hợp. Khi hoàn thành các bước này, bạn sẽ thấy kết quả như Hình 6.5.



Nói cách khác, sau khi quá trình chuyển đổi 3NF hoàn tất, cơ sở dữ liệu của bạn sẽ chứa bốn bảng:

PROJECT (**PROJ_NUM**, PROJ_NAME)

EMPLOYEE (**EMP_NUM**, EMP_NAME, JOB_CLASS)

JOB (**JOB_CLASS**, CHG_HOUR)

ASSIGNMENT (**PROJ_NUM**, **EMP_NUM**, ASSIGN_HOURS)

Lưu ý rằng điều này chuyển đổi đã loại bỏ sự phụ thuộc bậc cao của bảng EMPLOYEE ban đầu. Các bảng bây giờ được cho là ở dạng chuẩn thứ ba (3NF).

Lưu ý:

Một bảng ở dạng chuẩn thứ ba (3NF) khi:

- Nó ở dạng 2NF.
- và
- Nó không chứa các phụ thuộc bắc cầu.

Third Normal Form (3NF)

Một bảng được coi là 3NF khi nó đã đạt đến 2NF và không có thuộc tính không phải là thuộc tính khóa phụ thuộc vào một thuộc tính không phải là thuộc tính khóa khác; nghĩa là nó không thể bao gồm các phụ thuộc chuyển tiếp.

Thật thú vị khi lưu ý những điểm tương đồng giữa việc giải quyết các vấn đề 2NF và 3NF. Để chuyển đổi một bảng từ 1NF sang 2NF, cần loại bỏ các phụ thuộc một phần. Để chuyển đổi một bảng từ 2NF sang 3NF, cần phải loại bỏ các phụ thuộc bắc cầu. Bất kể phụ thuộc “có vấn đề” là phụ thuộc một phần hay phụ thuộc bắc cầu, giải pháp đều giống nhau: tạo một bảng mới cho từng phụ thuộc vấn đề. Yếu tố quyết định sự phụ thuộc của bài toán vẫn còn trong bảng gốc và được đặt làm khóa chính của bảng mới. Các phần phụ thuộc của sự phụ thuộc bài toán được loại bỏ khỏi bảng ban đầu và được đặt làm thuộc tính không chính thức trong bảng mới.

Tuy nhiên, hãy lưu ý rằng mặc dù kỹ thuật này giống nhau nhưng điều bắt buộc là 2NF phải được bãi bỏ trước khi chuyển sang 3NF; hãy chắc chắn giải quyết các phụ thuộc một phần trước khi giải quyết các phụ thuộc bắc cầu. Ngoài ra, hãy nhớ lại giả định đã được đưa ra khi bắt đầu thảo luận về chuẩn hóa—rằng mỗi bảng chỉ có một khóa dự tuyển, đó là khóa chính. Nếu một bảng có nhiều khóa ứng cử viên thì quy trình tổng thể vẫn giữ nguyên nhưng có những cân nhắc bổ sung.

Ví dụ: nếu một bảng có nhiều khóa ứng viên và một trong số đó là khóa tổng hợp thì bảng có thể có các phụ thuộc một phần dựa trên ứng cử viên tổng hợp này khóa, ngay cả khi khóa chính được chọn là một thuộc tính duy nhất. Trong những trường hợp đó, theo quy trình được mô tả ở trên, những phần phụ thuộc đó sẽ được coi là phần phụ thuộc bắc cầu và sẽ không được giải quyết cho đến khi 3NF. Quy trình đơn giản hóa được mô tả ở đây cho phép người thiết kế đạt được kết quả chính xác, nhưng thông qua thực hành, bạn nên nhận ra tất cả các khóa dự tuyển cũng như các phần phụ thuộc của chúng và giải quyết chúng một cách thích hợp. Sự tồn tại của nhiều khóa ứng viên cũng có thể ảnh hưởng đến việc xác định các phụ thuộc bắc cầu. Trước đây, sự phụ thuộc bắc cầu được xác định là tồn tại khi một thuộc tính không chính xác xác định một thuộc tính không chính khác. Khi có nhiều khóa ứng cử viên, việc xác định thuộc tính không chính thức là thuộc tính không thuộc bất kỳ khóa ứng viên nào là rất quan trọng. Nếu định thức của một sự phụ thuộc hàm không phải là khóa chính mà là một phần của khóa ứng viên khác thì nó không phải là thuộc tính không chính và không báo hiệu sự hiện diện của một phụ thuộc bắc cầu.

6-4 Cải thiện thiết kế

Bây giờ, các cấu trúc bảng đã được làm sạch để loại bỏ các phụ thuộc cục bộ và phụ thuộc bắc cầu rắc rối, bạn có thể tập trung vào việc cải thiện khả năng cung cấp thông tin của cơ sở dữ liệu và nâng cao các đặc tính hoạt động của nó. Trong một số đoạn tiếp theo, bạn sẽ

tìm hiểu về các loại vấn đề khác nhau mà bạn cần giải quyết để tạo ra một tập hợp bảng chuẩn hóa tốt. Để ngắn gọn, mỗi phần chỉ trình bày một ví dụ - người thiết kế phải áp dụng nguyên tắc này cho tất cả các bảng còn lại trong thiết kế. Hãy nhớ rằng bản thân việc chuẩn hóa không thể dựa vào để tạo ra các thiết kế tốt. Thay vào đó, việc chuẩn hóa có giá trị vì việc sử dụng nó giúp loại bỏ sự dư thừa dữ liệu.

Đánh giá các nhiệm vụ PK(Evaluate PK Assignments) Mỗi khi một nhân viên mới được nhập vào bảng EMPLOYEE, giá trị JOB_CLASS phải được nhập vào. Thật không may, rất dễ mắc lỗi nhập dữ liệu dẫn đến vi phạm tính toàn vẹn tham chiếu. Ví dụ: nhập *DB Designer* thay vì *Database Designer* cho thuộc tính JOB_CLASS trong bảng EMPLOYEE sẽ gây ra vi phạm như vậy. Do đó, sẽ tốt hơn nếu thêm thuộc tính JOB_CODE để tạo mã định danh duy nhất. Việc bổ sung thuộc tính JOB_CODE tạo ra sự phụ thuộc sau:

JOB_CODE → JOB_CLASS, CHG_HOUR

Nếu bạn cho rằng JOB_CODE là khóa chính thích hợp thì thuộc tính mới này sẽ tạo ra sự phụ thuộc sau:

JOB_CLASS → CHG_HOUR

Tuy nhiên, sự phụ thuộc này không phải là sự phụ thuộc bắc cầu vì định thức là một khóa ứng viên. Hơn nữa, sự hiện diện của JOB_CODE làm giảm đáng kể khả năng vi phạm tính toàn vẹn tham chiếu. Lưu ý rằng bảng JOB mới hiện có hai khóa ứng viên - JOB_CODE và JOB_CLASS. Trong trường hợp này, JOB_CODE là khóa chính được chọn cũng như khóa thay thế. Khóa thay thế, như bạn nên nhớ, là một PK nhân tạo được người thiết kế giới thiệu với mục đích đơn giản hóa việc gán khóa chính cho các bảng. Khóa thay thế thường là số, chúng thường được DBMS tạo tự động, chúng không có nội dung ngữ nghĩa (chúng không có ý nghĩa đặc biệt) và chúng thường bị ẩn khỏi người dùng cuối.

Đánh giá các quy ước đặt tên(Evaluate Naming Conventions) Tốt nhất là nên tuân thủ các quy ước đặt tên được nêu trong Chương 2, Mô hình dữ liệu. Do đó, CHG_HOUR sẽ được đổi thành JOB_CHG_HOUR để biểu thị mối liên hệ của nó với bảng JOB. Ngoài ra, tên thuộc tính JOB_CLASS không mô tả đầy đủ các mục như Systems Analyst, DatabaseDesigner, v.v.; nhãn JOB_DESCRIPTION phù hợp với các mục nhập hơn. Ngoài ra, bạn có thể nhận thấy rằng HOURS đã được thay đổi thành ASSIGN_HOURS khi chuyển đổi từ 1NF sang 2NF. Sự thay đổi đó cho phép bạn liên kết số giờ đã làm việc với bảng ASSIGNMENT.

Tinh chỉnh tính nguyên tử của thuộc tính(Refine Attribute Atomicity) Nói chung là một cách thực hành tốt để chú ý đến yêu cầu về *tính nguyên tử*. Thuộc tính nguyên tử là thuộc tính không thể chia nhỏ hơn nữa. Thuộc tính như vậy được cho là thể hiện tính nguyên tử. Rõ ràng, việc sử dụng EMP_NAME trong bảng EMPLOYEE không mang tính nguyên tử vì EMP_NAME có thể được phân tách thành họ, tên và chữ cái đầu. Bằng cách cải thiện mức độ nguyên tử, bạn cũng có được tính linh hoạt khi truy vấn. Ví dụ: nếu bạn sử dụng EMP_LNAME, EMP_FNAME và EMP_INITIAL, bạn có thể dễ dàng tạo danh sách điện thoại bằng cách sắp xếp họ, tên và tên viết tắt. Nhiệm vụ như vậy sẽ rất khó khăn nếu các thành phần tên nằm trong một thuộc tính duy nhất. Nhìn chung, các nhà thiết kế

thích sử dụng các thuộc tính đơn giản, có giá trị đơn, như được chỉ ra bởi các quy tắc nghiệp vụ và yêu cầu xử lý.

Xác định các thuộc tính mới (*Identify New Attributes*) Nếu bảng EMPLOYEE được sử dụng trong môi trường thực tế thì một số thuộc tính khác sẽ phải được thêm vào. Ví dụ: các khoản thanh toán tổng

tiền lương hàng năm, các khoản thanh toán An sinh xã hội và các khoản thanh toán Medicare sẽ là điều mong muốn. Thuộc tính ngày thuê nhân viên (EMP_HIREDATE) có thể được sử dụng để theo dõi thời gian làm việc của nhân viên và nó có thể dùng làm cơ sở cho thưởng cho nhân viên làm việc lâu dài và các biện pháp nâng cao tinh thần khác. Nguyên tắc tương tự phải được áp dụng cho tất cả các bảng khác trong thiết kế của bạn.

Xác định các mối quan hệ mới (*Identify New Relationships*) Theo báo cáo ban đầu, người dùng cần theo dõi nhân viên nào đang đóng vai trò là người quản lý của từng dự án. Điều này có thể được thực hiện dưới dạng mối quan hệ giữa EMPLOYEE và PROJECT. Từ báo cáo ban đầu, rõ ràng mỗi dự án chỉ có một người quản lý. Do đó, khả năng cung cấp thông tin chi tiết về từng người quản lý dự án của hệ thống được đảm bảo bằng cách sử dụng EMP_NUM làm khóa ngoại trong PROJECT. Hành động đó đảm bảo rằng bạn có thể truy cập chi tiết dữ liệu người quản lý của từng PROJECT mà không tạo ra sự trùng lặp dữ liệu không cần thiết và không mong muốn. Người thiết kế phải cẩn thận đặt đúng thuộc tính vào đúng bảng bằng cách sử dụng các nguyên tắc chuẩn hóa.

Tinh chỉnh Khóa chính theo yêu cầu đối với Độ chi tiết của dữ liệu (*Refine Primary Keys as Required for Data*

***Granularity*)** Đề cập đến mức độ chi tiết được biểu thị bằng các giá trị được lưu trữ trong hàng của bảng. Dữ liệu được lưu trữ ở mức độ chi tiết thấp nhất được gọi là *dữ liệu nguyên tử*, như đã giải thích trước đó. Trong Hình 6.5, bảng ASSIGNMENT trong 3NF sử dụng thuộc tính ASSIGN_HOURS để biểu thị số giờ làm việc của một nhân viên nhất định trong một dự án nhất định. Tuy nhiên, những giá trị đó có được ghi lại ở mức độ chi tiết thấp nhất không? Nói cách khác, ASSIGN_HOURS có đại diện cho tổng *theo giờ*, tổng *theo ngày*, tổng *theo tuần*, tổng *theo tháng* hay tổng *theo năm*? Rõ ràng, ASSIGN_HOURS yêu cầu định nghĩa cẩn thận hơn. Trong trường hợp này, câu hỏi liên quan sẽ như sau: đối với khung thời gian nào - giờ, ngày, tuần, tháng, v.v. - bạn có muốn ghi lại dữ liệu ASSIGN_HOURS không?

Ví dụ: giả sử rằng sự kết hợp của EMP_NUM và PROJ_NUM là có thể chấp nhận được (composite) khóa chính trong bảng ASSIGNMENT. Khóa chính đó chỉ hữu ích khi biểu thị tổng số giờ mà một nhân viên đã làm việc trong một dự án kể từ khi bắt đầu. Việc sử dụng khóa chính thay thế như ASSIGN_NUM mang lại mức độ chi tiết thấp hơn và mang lại tính linh hoạt cao hơn. Ví dụ: giả sử rằng kết hợp

Atomic attribute

(thuộc tính nguyên tử)
Một thuộc tính không thể chia nhỏ hơn nữa để tạo ra các thành phần có ý nghĩa. Ví dụ: thuộc tính họ của một người không thể được chia nhỏ một cách có ý nghĩa.

atomicity (tính nguyên tử)

Không thể chia thành các đơn vị nhỏ hơn.

granularity (độ chi tiết)

Mức độ chi tiết được biểu thị bằng các giá trị được lưu trữ trong hàng của bảng. Dữ liệu được lưu trữ ở mức độ chi tiết thấp nhất được gọi là dữ liệu nguyên tử.

EMP_NUM và PROJ_NUM được sử dụng làm khóa chính, sau đó nhân viên tạo hai mục nhập “giờ làm việc” trong bảng ASSIGNMENT. Hành động đó vi phạm yêu cầu về tính toàn vẹn của thực thể. Ngay cả khi bạn thêm ASSIGN_DATE như một phần của PK tổng hợp, vi phạm tính toàn vẹn của thực thể vẫn được tạo ra nếu bất kỳ nhân viên nào thực hiện hai mục nhập trở lên cho cùng một dự án trong cùng một ngày. (Nhân viên có thể đã làm việc với dự án trong vài giờ vào buổi sáng và sau đó lại tiếp tục làm việc đó trong ngày.) Việc nhập dữ liệu tương tự sẽ không có vấn đề gì khi sử dụng ASSIGN_NUM làm khóa chính.

Lưu ý:

Trong thiết kế cơ sở dữ liệu lý tưởng, mức độ chi tiết mong muốn sẽ được xác định trong quá trình thiết kế khái niệm hoặc trong khi các yêu cầu đang được thu thập. Tuy nhiên, như bạn đã thấy trong chương này, nhiều thiết kế cơ sở dữ liệu liên quan đến việc sàng lọc các yêu cầu dữ liệu hiện có, do đó dẫn đến việc sửa đổi thiết kế. Trong môi trường thực tế, việc thay đổi các yêu cầu về độ chi tiết có thể dẫn đến những thay đổi trong việc lựa chọn khóa chính và những thay đổi đó cuối cùng có thể yêu cầu sử dụng các khóa thay thế.

Duy trì độ chính xác lịch sử (*Maintain Historical Accuracy*) Việc ghi phí công việc mỗi giờ vào bảng ASSIGNMENT là rất quan trọng để duy trì độ chính xác lịch sử của dữ liệu trong bảng. Sẽ là phù hợp nếu đặt tên cho thuộc tính này là ASSIGN_CHG_HOUR. Mặc dù thuộc tính này dường như có cùng giá trị với JOB_CHG_HOUR, nhưng điều này *chỉ đúng* nếu giá trị JOB_CHG_HOUR giữ nguyên mãi mãi. Thật hợp lý khi cho rằng mức lương công việc mỗi giờ sẽ thay đổi theo thời gian. Tuy nhiên, giả sử rằng các khoản phí cho từng dự án đã được tính toán và lập hóa đơn bằng cách nhân số giờ làm việc từ bảng ASSIGNMENT với mức phí mỗi giờ từ bảng JOB. Những khoản phí đó sẽ luôn hiển thị mức phí hiện tại theo giờ được lưu trữ trong bảng JOB thay vì mức phí theo giờ có hiệu lực tại thời điểm phân công.

Đánh giá bằng cách sử dụng thuộc tính phái sinh (*Evaluate Using Derived Attributes*) Cuối cùng, bạn có thể sử dụng thuộc tính phái sinh trong bảng ASSIGNMENT để lưu trữ chi phí thực tế được thực hiện cho một dự án. Thuộc tính dẫn xuất đó, được đặt tên là ASSIGN_CHARGE, là kết quả của việc nhân ASSIGN_HOURS với ASSIGN_CHG_HOUR. Điều này tạo ra sự phụ thuộc bắc cầu sao cho:

$$(\text{ASSIGN_CHARGE} + \text{ASSIGN_HOURS}) \rightarrow \text{ASSIGN_CHG_HOUR}$$

Từ quan điểm chức năng hệ thống, các giá trị thuộc tính dẫn xuất như vậy có thể được tính toán khi cần để viết báo cáo hoặc hóa đơn. Tuy nhiên, việc lưu trữ thuộc tính dẫn xuất trong bảng giúp dễ dàng viết phần mềm ứng dụng để tạo ra kết quả mong muốn. Ngoài ra, nếu nhiều giao dịch phải được báo cáo và/hoặc tóm tắt thì tính sẵn có của thuộc tính dẫn xuất sẽ tiết kiệm thời gian báo cáo. (Nếu phép tính được thực hiện tại thời điểm nhập dữ liệu, nó sẽ được hoàn thành khi người dùng cuối nhấn phím Enter, do đó đầy nhanh quá

trình.) Xem lại Chương 4 để thảo luận về ý nghĩa của việc lưu trữ các thuộc tính dẫn xuất trong bảng cơ sở dữ liệu. Những cải tiến được mô tả trong các phần trước được minh họa trong các bảng và sơ đồ phụ thuộc được hiển thị trong Hình 6.6.

FIGURE 6.6 THE COMPLETED DATABASE

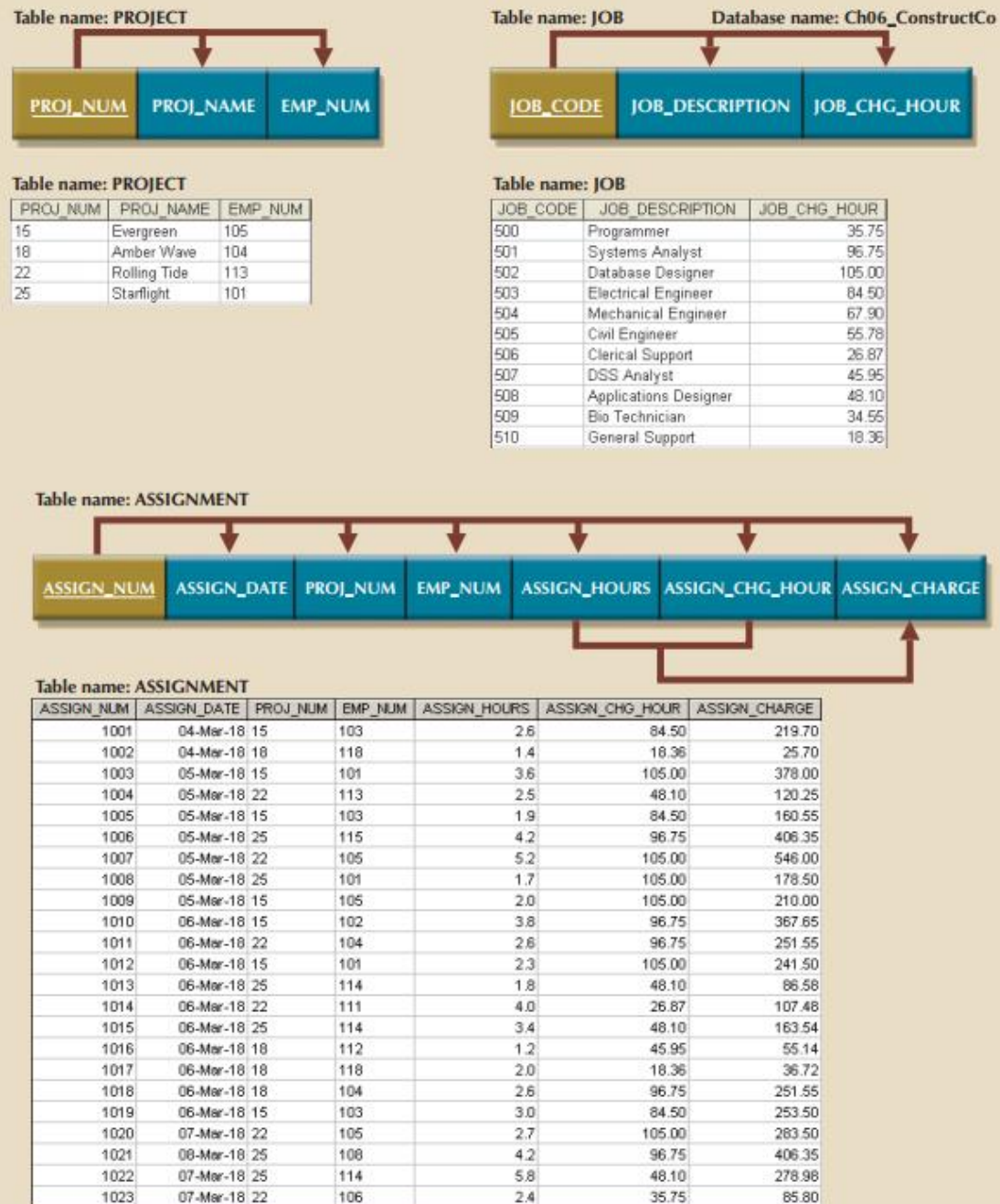


FIGURE 6.6 THE COMPLETED DATABASE (CONTINUED)



Hình 6.6 là một cải tiến vượt bậc so với thiết kế cơ sở dữ liệu ban đầu. Nếu phần mềm ứng dụng được thiết kế đúng cách, bảng hoạt động nhiều nhất (ASSIGNMENT) chỉ yêu cầu nhập các giá trị PROJ_NUM, EMP_NUM và ASSIGN_HOURS. Ứng dụng có thể tạo các giá trị cho thuộc tính ASSIGN_NUM và ASSIGN_DATE. Ví dụ: ASSIGN_NUM có thể được tạo bằng cách sử dụng bộ đếm và ASSIGN_DATE có thể là ngày hệ thống được ứng dụng đọc và tự động được nhập vào bảng ASSIGNMENT. Ngoài ra, phần mềm ứng dụng có thể tự động chèn giá trị ASSIGN_CHG_HOUR chính xác bằng cách ghi giá trị JOB_CHG_HOUR của bảng JOB thích hợp vào bảng ASSIGNMENT. (Các bảng JOB và ASSIGNMENT có liên quan với nhau thông qua thuộc tính JOB_CODE.) Nếu giá trị JOB_CHG_HOUR của bảng JOB thay đổi, lần chèn giá trị đó vào bảng ASSIGNMENT tiếp theo sẽ tự động phản ánh sự thay đổi đó. Cấu trúc bảng do đó giảm thiểu nhu cầu can thiệp của con người. Trên thực tế, nếu hệ thống yêu cầu nhân viên nhập giờ làm việc của riêng họ, họ có thể quét EMP_NUM của mình vào bảng ASSIGNMENT bằng cách sử dụng đầu đọc thẻ từ để nhập danh tính của họ. Do đó, cấu trúc của bảng ASSIGNMENT có thể tạo tiền đề cho việc duy trì một số mức độ bảo mật mong muốn.

6-5. Người thay thế Những cân nhắc chính

Mặc dù thiết kế này đáp ứng các yêu cầu về thực thể quan trọng và tính toàn vẹn tham chiếu, người thiết kế vẫn phải giải quyết một số mối quan tâm. Ví dụ: khóa chính tổng hợp có thể trở nên quá cồng kềnh để sử dụng khi số lượng thuộc tính tăng lên. (Việc tạo khóa ngoại phù hợp trở nên khó khăn khi bảng liên quan sử dụng khóa chính tổng hợp. Ngoài ra, khóa chính tổng hợp khiến việc viết quy trình tìm kiếm trở nên khó khăn hơn.) Hoặc, một thuộc tính khóa chính có thể đơn giản là có quá nhiều nội dung mô tả, có thể sử dụng được - đó là lý do tại sao thuộc tính JOB_CODE được thêm vào bảng JOB để dùng làm

khóa chính của nó. Khi khóa chính được coi là không phù hợp vì lý do nào đó, các nhà thiết kế sẽ sử dụng khóa thay thế, như đã thảo luận ở chương trước.

Ở cấp độ triển khai, khóa thay thế là thuộc tính do hệ thống xác định thường được tạo và quản lý thông qua DBMS. Thông thường, khóa thay thế do hệ thống xác định là số và giá trị của nó sẽ tự động tăng lên cho mỗi hàng mới. Ví dụ: Microsoft Access sử dụng kiểu dữ liệu AutoNumber, Microsoft SQL Server sử dụng cột nhận dạng và Oracle sử dụng đối tượng chuỗi.

Hãy nhớ lại Phần 6-4 rằng thuộc tính JOB_CODE được chỉ định làm khóa chính của bảng JOB. Tuy nhiên, hãy nhớ rằng thuộc tính JOB_CODE không ngăn chặn các mục nhập trùng lặp, như được hiển thị trong bảng JOB ở Bảng 6.4.

TABLE 6.4		
DUPLICATE ENTRIES IN THE JOB TABLE		
JOB_CODE	JOB_DESCRIPTION	JOB_CHG_HOUR
511	Programmer	\$ 35.75
512	Programmer	\$ 35.75

Rõ ràng, các mục nhập dữ liệu trong Bảng 6.4 là không phù hợp vì chúng trùng lặp các bản ghi hiện có—tuy nhiên, không có sự vi phạm nào về tính toàn vẹn của thực thể hoặc tính toàn vẹn tham chiếu. Vấn đề về nhiều bản ghi trùng lặp này đã được tạo ra khi thuộc tính JOB_CODE được thêm làm PK. (Khi JOB_DESCRIPTION ban đầu được chỉ định là PK, DBMS sẽ đảm bảo các giá trị duy nhất cho tất cả các mục mô tả công việc khi nó được yêu cầu thực thi tính toàn vẹn của thực thể. Tuy nhiên, tùy chọn đó đã tạo ra các vấn đề gây ra việc sử dụng thuộc tính JOB_CODE ngay từ đầu!) Trong mọi trường hợp, nếu JOB_CODE là PK thay thế, bạn vẫn phải đảm bảo sự tồn tại của các giá trị duy nhất trong JOB_DESCRIPTION *thông qua việc sử dụng chỉ mục duy nhất*.

Lưu ý rằng tất cả các bảng còn lại (PROJECT, ASSIGNMENT, và EMPLOYEE) đều có cùng giới hạn. Ví dụ: nếu bạn sử dụng thuộc tính EMP_NUM trong bảng EMPLOYEE làm PK, bạn có thể tạo nhiều mục nhập cho cùng một nhân viên. Để tránh vấn đề đó, bạn có thể tạo một chỉ mục duy nhất cho EMP_LNAME, EMP_FNAME và EMP_INITIAL, nhưng sau đó bạn sẽ làm thế nào giải quyết hai nhân viên tên Joe B. Smith? Trong trường hợp đó, bạn có thể sử dụng thuộc tính khác (tốt nhất là được xác định từ bên ngoài) để làm cơ sở cho một chỉ mục duy nhất.

Cần nhắc lại rằng thiết kế cơ sở dữ liệu thường liên quan đến sự đánh đổi và việc thực hiện các phán đoán chuyên môn. Trong môi trường thực tế, bạn phải đạt được sự cân bằng giữa tính toàn vẹn của thiết kế và tính linh hoạt. Ví dụ: bạn có thể thiết kế bảng ASSIGNMENT để sử dụng chỉ mục duy nhất trên PROJ_NUM, EMP_NUM và ASSIGN_DATE nếu bạn muốn giới hạn một nhân viên chỉ có một mục nhập ASSIGN_HOURS mỗi ngày. Hạn chế đó sẽ đảm bảo rằng nhân viên không thể nhập cùng một số giờ nhiều lần trong bất kỳ ngày cụ thể nào. Thật không may, hạn chế đó có thể là điều không mong muốn theo quan điểm của người quản lý. Suy cho cùng, nếu một nhân viên làm việc nhiều lần trong một dự án trong bất kỳ ngày nhất định nào, thì phải có thể tạo nhiều mục nhập cho cùng một nhân viên và cùng một dự án trong ngày đó. Trong trường

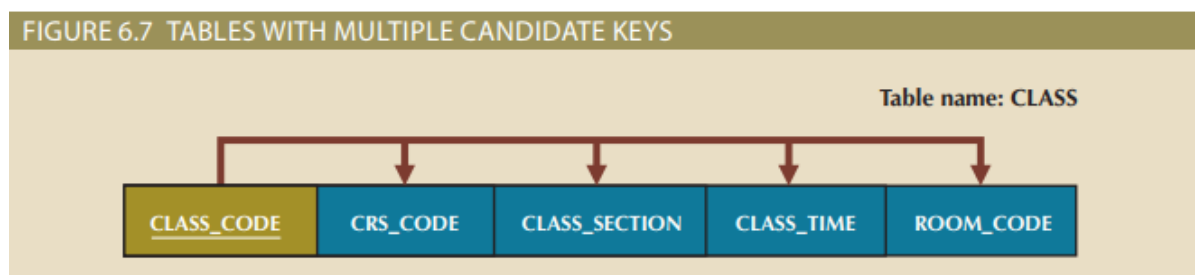
hợp đó, giải pháp tốt nhất có thể là thêm một thuộc tính mới được xác định bên ngoài - chẳng hạn như sơ khai, chứng từ hoặc số vé - để đảm bảo tính duy nhất. Trong mọi trường hợp, việc kiểm tra dữ liệu thường xuyên sẽ phù hợp.

6-6. Bảng biểu mẫu thông thường cấp cao hơn

Các bảng trong 3NF sẽ hoạt động phù hợp trong cơ sở dữ liệu giao dịch nghiệp vụ. Tuy nhiên, các dạng chuẩn cao hơn đôi khi cũng hữu ích. Trong phần này, bạn sẽ tìm hiểu về trường hợp đặc biệt của 3NF, được gọi là dạng chuẩn Boyce-Codd và về dạng chuẩn 4 (4NF).

6-6a. Dạng chuẩn Boyce-Codd

Một bảng có **boyce-codd normal form** (dạng chuẩn boyce-codd) (BCNF) khi mọi định thức trong bảng là một khóa ứng viên. (Hãy nhớ lại trong Chương 3 rằng một khóa dự tuyển có các đặc điểm giống như khóa chính, nhưng vì lý do nào đó, nó không được chọn làm khóa chính.) Rõ ràng, khi một bảng chỉ chứa một khóa dự tuyển thì 3NF và BCNF là tương đương. Nói cách khác, BCNF chỉ có thể bị vi phạm khi bảng chứa nhiều hơn một khóa ứng viên. Trong các ví dụ về dạng chuẩn trước đó, các bảng chỉ có một khóa dự tuyển được sử dụng để đơn giản hóa việc giải thích. Tuy nhiên, hãy nhớ rằng luôn có thể có nhiều khóa ứng viên và các quy tắc chuẩn hóa tập trung vào các khóa ứng viên chứ không chỉ khóa chính. Hãy xem xét cấu trúc bảng được hiển thị trong Hình 6.7.



Bảng CLASS có hai khóa dự tuyển:

CLASS_CODE

CRS_CODE + CLASS_SECTION

Bảng ở dạng 1NF vì các thuộc tính khóa được xác định và tất cả các thuộc tính không khóa được xác định bởi khóa. Điều này đúng cho cả hai khóa ứng cử viên. Cả hai khóa ứng viên đều đã được xác định và tất cả các thuộc tính khác có thể được xác định bằng khóa ứng viên. Bảng ở dạng 2NF vì nó ở dạng 1NF và không có sự phụ thuộc một phần nào vào khóa ứng viên. Vì CLASS_CODE là khóa dự tuyển thuộc tính duy nhất nên vấn đề phụ thuộc một phần không được áp dụng. Tuy nhiên, khóa ứng viên tổng hợp của CRS_CODE + CLASS_SECTION có thể có sự phụ thuộc một phần, do đó, 2NF phải được đánh giá cho khóa ứng viên đó. Trong trường hợp này, không có sự phụ thuộc một phần nào liên quan đến khóa tổng hợp. Cuối cùng, bảng ở dạng 3NF vì không có sự phụ thuộc bắc cầu. Hãy nhớ rằng, vì CRS_CODE + CLASS_SECTION là khóa ứng viên nên thực tế là tổ hợp này có thể xác định CLASS_TIME và ROOM_CODE không phải là sự phụ thuộc

bắc cầu. Sự phụ thuộc bắc cầu tồn tại khi một thuộc tính *không khóa* có thể xác định một thuộc tính không khóa khác và CRS_CODE + CLASS_SECTION là khóa.

Lưu ý:

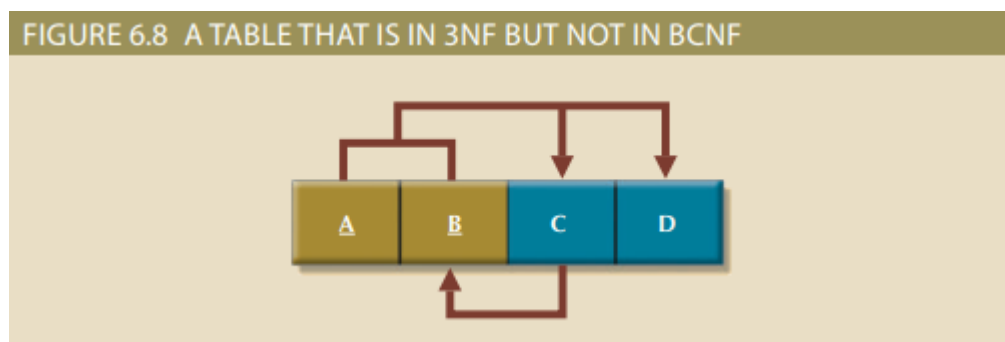
Một bảng ở dạng chuẩn Boyce-Codd (BCNF) khi mọi định thức trong bảng đều là một khóa ứng viên

boyce-codd normal form
(dạng chuẩn boyce-codd)
(BCNF)

Một loại đặc biệt của dạng chuẩn thứ ba (3NF) trong đó mỗi định thức là một khóa ứng viên. Một bảng trong BCNF phải ở dạng 3NF. Xem thêm xác định

Hầu hết các nhà thiết kế coi BCNF là trường hợp đặc biệt của 3NF. Trong thực tế, nếu sử dụng các kỹ thuật trình bày trong chương này thì hầu hết các bảng đều tuân thủ các yêu cầu BCNF khi đạt đến 3NF. Vậy làm thế nào một bảng có thể ở dạng 3NF mà không ở dạng BCNF? Để trả lời câu hỏi đó, bạn phải nhớ rằng sự phụ thuộc bắc cầu tồn tại khi một thuộc tính không chính phụ thuộc vào một thuộc tính không chính khác.

Nói cách khác, một bảng ở dạng 3NF khi nó ở dạng 2NF và không có sự phụ thuộc bắc cầu, nhưng còn trường hợp trong đó một thuộc tính khóa là yếu tố quyết định của một thuộc tính khóa khác thì sao? Điều kiện đó không vi phạm 3NF, tuy nhiên nó không đáp ứng được các yêu cầu của BCNF (xem Hình 6.8) vì BCNF yêu cầu mọi định thức trong bảng phải là một khóa dự tuyển.



Lưu ý các phụ thuộc chức năng này trong Hình 6.8:

$$A + B \rightarrow C, D$$

$$A + C \rightarrow B, D$$

$$C \rightarrow B$$

Lưu ý rằng cấu trúc này có hai khóa ứng viên: (A + B) và (A + C). Cấu trúc bảng được hiển thị trong Hình 6.8 không có sự phụ thuộc một phần và cũng không chứa các phụ thuộc bắc cầu. (Điều kiện $C \rightarrow B$ chỉ ra rằng *một thuộc tính khóa xác định một phần của khóa chính* - và sự phụ thuộc đó *không* mang tính bắc cầu hay một phần vì thuộc tính phụ thuộc là thuộc tính nguyên tố!) Do đó, cấu trúc bảng trong Hình 6.8 đáp ứng các yêu cầu 3NF, mặc dù điều kiện $C \rightarrow B$ khiến bảng không đáp ứng được yêu cầu BCNF.

Để chuyển đổi cấu trúc bảng trong Hình 6.8 thành các cấu trúc bảng trong 3NF và trong BCNF, trước tiên hãy thay đổi khóa chính thành A + C. Sự thay đổi này là phù hợp vì sự

phụ thuộc $C \rightarrow B$ có nghĩa là C thực sự là một tập hợp con của B. Tại thời điểm này, bảng ở dạng 1NF vì nó chứa một phần phụ thuộc, $C \rightarrow B$. Tiếp theo, thực hiện theo các quy trình phân tách tiêu chuẩn để tạo ra các kết quả được hiển thị trong Hình 6.9.

Để xem quy trình này có thể được áp dụng như thế nào cho một vấn đề thực tế, hãy kiểm tra dữ liệu mẫu trong Bảng 6.5.

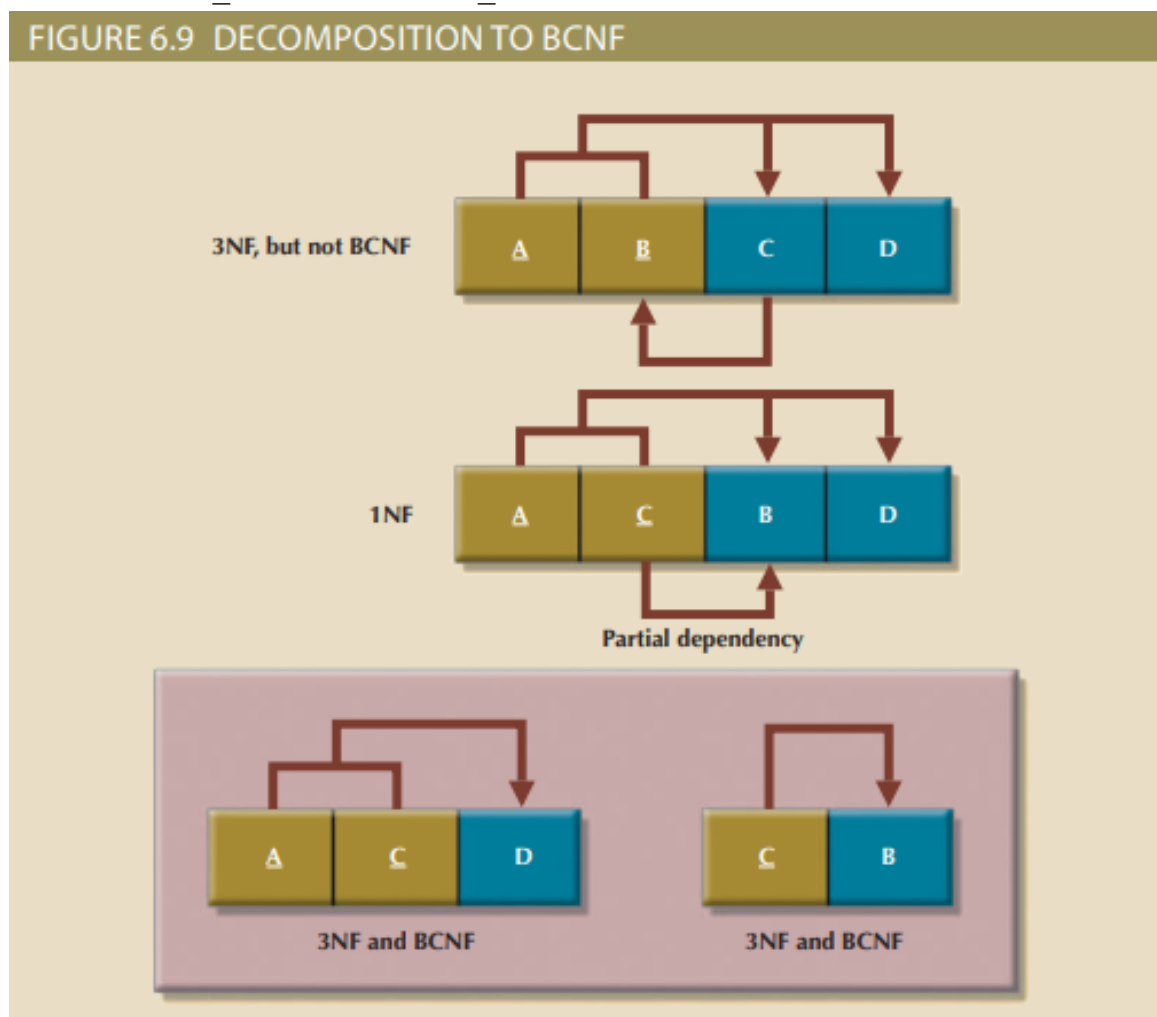
Bảng 6.5 phản ánh các điều kiện sau:

- Mỗi CLASS_CODE xác định một lớp duy nhất. Điều kiện này minh họa trường hợp một khóa học có thể tạo ra nhiều lớp. Ví dụ: một khóa học có nhãn INFS420 có thể được dạy thành hai lớp (phần), mỗi lớp được xác định bằng một mã duy nhất để hỗ trợ đăng ký. Do đó, CLASS_CODE 32456 có thể xác định INFS 420, phần 1, trong khi CLASS_CODE 32457 có thể xác định INFS 420, phần lớp 2. Hoặc CLASS_CODE 28458 có thể xác định QM 362, phần lớp 5.
- Một học sinh có thể học nhiều lớp. Ví dụ, hãy lưu ý rằng học sinh 125 đã thi cả 21334 và 32456, đạt điểm A và C tương ứng.
- Một giáo viên có thể dạy nhiều lớp nhưng mỗi lớp chỉ có một giáo viên dạy. Lưu ý rằng nhân viên 20 dạy các lớp được xác định là 32456 và 28458.

Cấu trúc hiển thị trong Bảng 6.5 được phản ánh trong Bảng A của Hình 6.10:

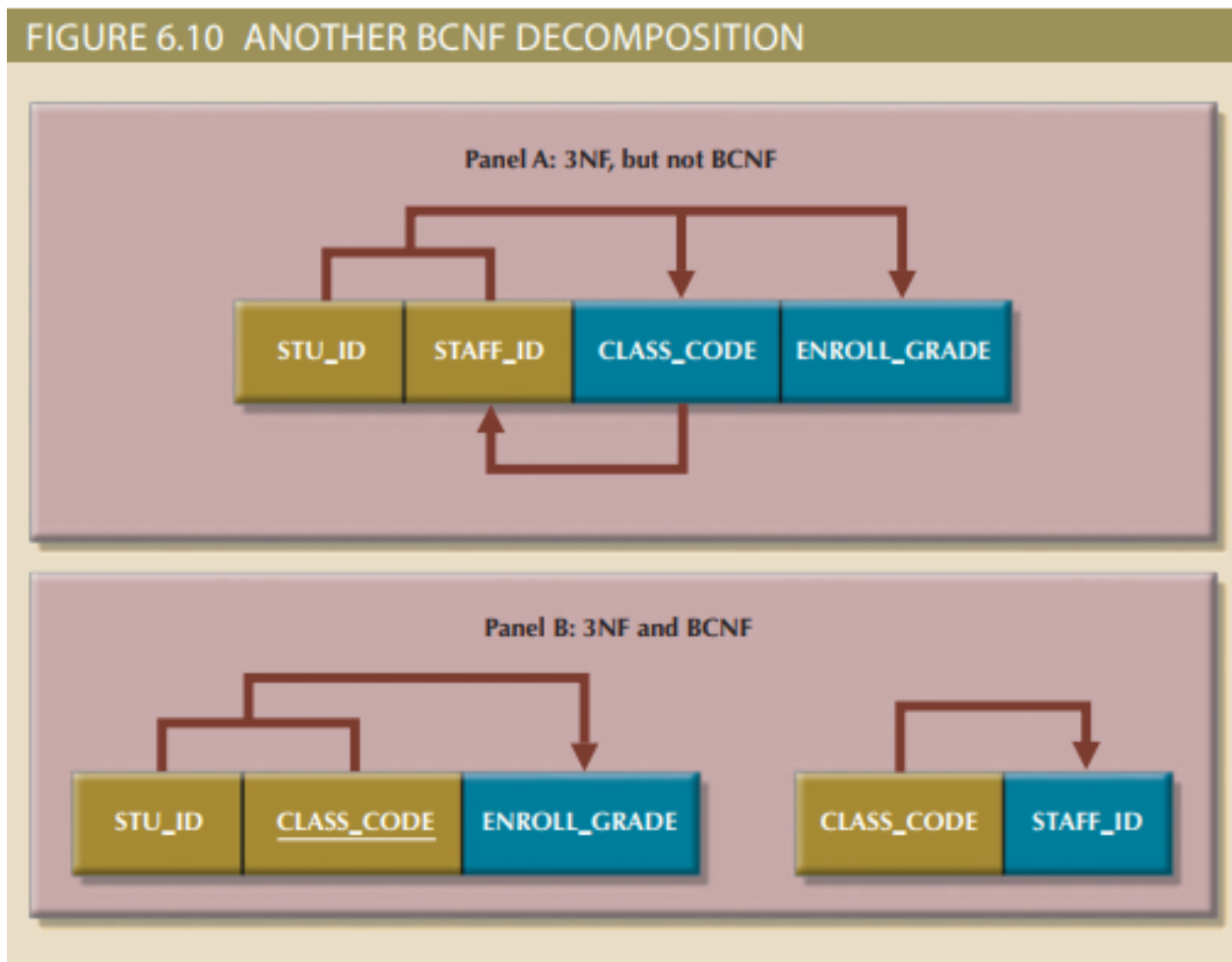
$STU_ID + STAFF_ID \rightarrow CLASS_CODE, ENROLL_GRADE$

$CLASS_CODE \rightarrow STAFF_ID$



TALLE 6.5			
SAMPLE DATA FOR A BCNF CONVERSION			
STU_ID	STAFF_ID	CLASS_CODE	ENROLL_GRADE
125	25	21334	A
125	20	32456	C
135	20	28458	B
144	25	27563	C
144	20	32456	B

Phần A của Hình 6.10 cho thấy một cấu trúc rõ ràng là ở dạng 3NF, nhưng bảng được biểu thị bằng cấu trúc này có một vấn đề lớn vì nó đang cố gắng mô tả hai điều: phân công nhân viên cho các lớp và thông tin tuyển sinh của học sinh. Cấu trúc bảng có mục đích kép như vậy sẽ gây ra sự bất thường. Ví dụ: nếu một nhân viên khác được phân công dạy lớp 32456, hai hàng sẽ yêu cầu cập nhật, do đó tạo ra cập nhật bất thường. Ngoài ra, nếu học sinh 135 bỏ lớp 28458, thông tin về người dạy lớp đó sẽ bị mất, do đó tạo ra hiện tượng xóa bất thường. Giải pháp cho vấn đề này là phân rã cấu trúc bảng, theo quy trình đã nêu trước đó. Việc phân tách Bảng B được hiển thị trong Hình 6.10 mang lại hai cấu trúc bảng phù hợp với cả yêu cầu 3NF và BCNF.



Hãy nhớ rằng một bảng nằm trong BCNF khi mọi định thức trong bảng đó đều là một khóa dự tuyển. Do đó, khi một bảng chỉ chứa một khóa dự tuyển thì 3NF và BCNF là tương đương.

6-6b. **Dạng chuẩn thứ tư (4NF)**

Bạn có thể gặp phải cơ sở dữ liệu được thiết kế kém hoặc bạn có thể được yêu cầu chuyển đổi bảng tính sang định dạng cơ sở dữ liệu trong đó tồn tại nhiều thuộc tính đa giá trị. Ví dụ, hãy xem xét khả năng một nhân viên có thể đảm nhiệm nhiều nhiệm vụ và cũng có thể tham gia vào nhiều tổ chức dịch vụ. Giả sử nhân viên 10123 tình nguyện viên cho the Red Cross và United Way. Ngoài ra, cùng một nhân viên có thể được phân công làm việc trong ba dự án: 1, 3 và 4. Hình 6.11 minh họa cách ghi lại các sự kiện đó theo những cách rất khác nhau.

FIGURE 6.11 TABLES WITH MULTIVALUED DEPENDENCIES

Database name: Ch06_Service

Table name: VOLUNTEER_V1

EMP_NUM	ORG_CODE	ASSIGN_NUM
10123	RC	1
10123	UW	3
10123		4

Table name: VOLUNTEER_V2

EMP_NUM	ORG_CODE	ASSIGN_NUM
10123	RC	
10123	UW	
10123		1
10123		3
10123		4

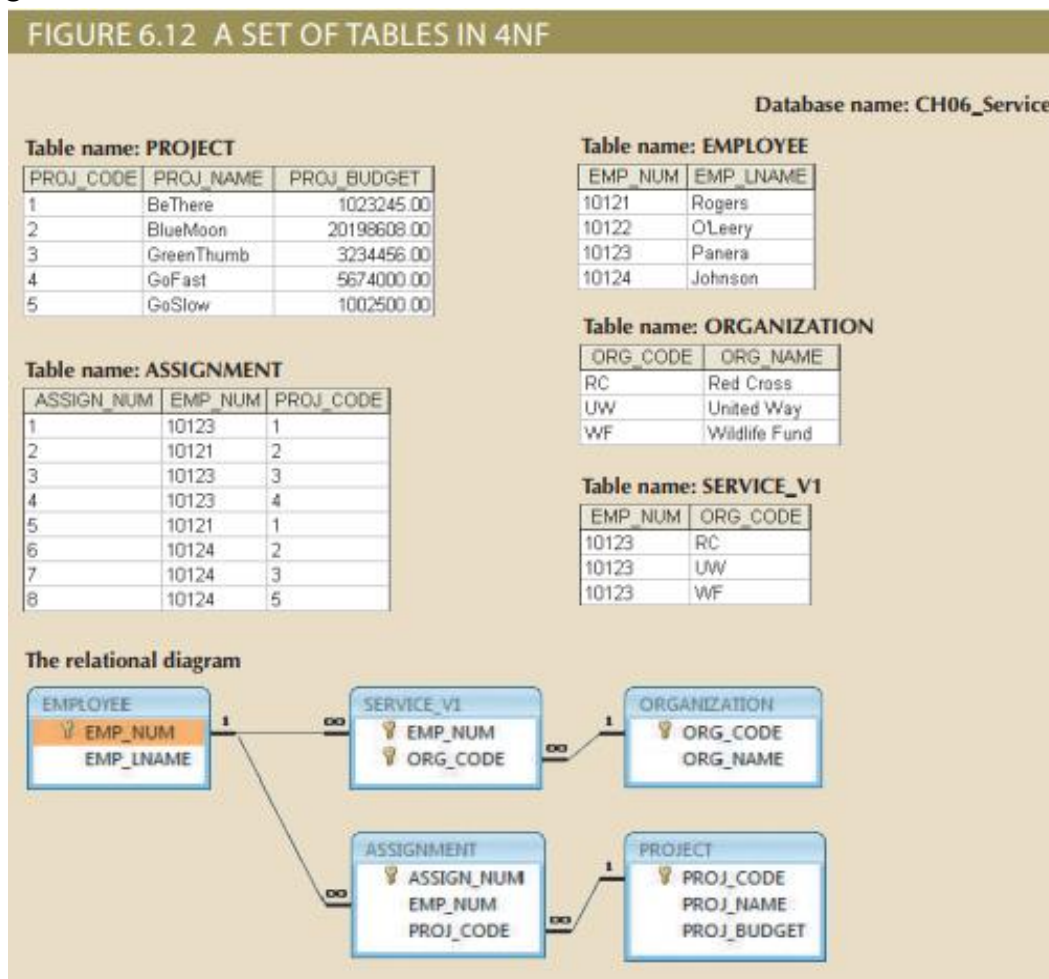
Table name: VOLUNTEER_V3

EMP_NUM	ORG_CODE	ASSIGN_NUM
10123	RC	1
10123	RC	3
10123	UW	4

Có vấn đề với các bảng trong Hình 6.11. Mỗi thuộc tính `ORG_CODE` và `ASSIGN_NUM` có thể có nhiều giá trị khác nhau. Trong thuật ngữ chuẩn hóa, tình huống này được gọi là sự phụ thuộc đa giá trị, xảy ra khi một khóa xác định nhiều giá trị của hai thuộc tính khác và các thuộc tính đó độc lập với nhau. (Một nhân viên có thể có nhiều mục nhập dịch vụ và nhiều mục nhập phân công. Do đó, một `EMP_NUM` có thể xác định nhiều giá trị của `ORG_CODE` và nhiều giá trị của `ASSIGN_NUM`; tuy nhiên, `ORG_CODE` và `ASSIGN_NUM` độc lập với nhau.) Sự hiện diện của một phụ thuộc đa giá trị có nghĩa là nếu chuyển đổi bảng 1 và 2 được triển khai, các bảng có thể chứa khá nhiều giá trị null; trên thực tế, các bảng thậm chí không có khóa ứng viên khả thi. (Các giá trị `EMP_NUM` không phải là duy nhất, vì vậy chúng không thể là PKs. Không thể sử dụng kết hợp các thuộc tính trong phiên bản bảng 1 và 2 để tạo PK vì một số trong chúng chứa giá trị rỗng.) Điều kiện như vậy là không mong muốn, đặc biệt khi có hàng nghìn nhân viên, nhiều người trong số họ có thể đảm nhiệm nhiều công việc và nhiều hoạt động dịch vụ. Phiên bản 3 ít nhất có PK, nhưng nó bao gồm tất cả các thuộc tính trong bảng. Trên thực tế, phiên bản 3 đáp ứng các yêu cầu 3NF, tuy nhiên nó chứa nhiều phần dư thừa rõ ràng là không mong muốn.

Giải pháp là loại bỏ các vấn đề do sự phụ thuộc đa giá trị gây ra. Bạn thực hiện việc này bằng cách tạo các bảng mới cho các thành phần của phần phụ thuộc đa giá trị. Trong ví dụ này, sự phụ thuộc đa giá trị được giải quyết và loại bỏ bằng cách tạo các bảng

ASSIGNMENT và SERVICE_V1 được mô tả trong Hình 6.12. Những bảng đó được cho là ở dạng 4NF.



Nếu bạn tuân theo các quy trình thiết kế phù hợp được minh họa trong cuốn sách này, bạn sẽ không gặp phải vấn đề như trong Hình 6.11. Cụ thể, cuộc thảo luận về 4NF phần lớn mang tính học thuật nếu bạn đảm bảo rằng các bảng của mình tuân thủ hai quy tắc sau:

1. Tất cả các thuộc tính phải phụ thuộc vào khóa chính nhưng phải độc lập với nhau.
2. Không có hàng nào có thể chứa hai hoặc nhiều thông tin đa giá trị về một thực thể.

fourth normal form (4NF)

(dạng chuẩn thứ tư)

Một bảng ở dạng 4NF nếu nó ở dạng 3NF và không chứa nhiều tập hợp phụ thuộc đa giá trị độc lập.

Lưu ý:

Bảng ở **fourth normal form** (dạng chuẩn thứ tư) (4NF) khi nó ở dạng 3NF và không có phụ đa giá trị.

6-7. Chuẩn hóa và thiết kế cơ sở dữ liệu

Các bảng trong Hình 6.6 minh họa cách thức các thủ tục chuẩn hóa có thể được sử dụng để tạo ra các bảng tốt từ các bảng kém. Bạn có thể sẽ có nhiều cơ hội để áp dụng kỹ năng này vào thực tế khi bắt đầu làm việc với cơ sở dữ liệu trong thế giới thực. *Chuẩn hóa phải là một phần của quá trình thiết kế*. Do đó, hãy đảm bảo rằng các thực thể được đề xuất đáp ứng dạng chuẩn được yêu cầu *trước khi* tạo cấu trúc bảng. Hãy nhớ rằng nếu bạn làm theo các quy trình thiết kế được thảo luận trong Chương 3 và 4, khả năng xảy ra dị thường về dữ liệu sẽ rất nhỏ. Tuy nhiên, ngay cả những nhà thiết kế cơ sở dữ liệu giỏi nhất cũng thỉnh thoảng mắc phải những lỗi được phát hiện trong quá trình kiểm tra chuẩn hóa. Ngoài ra, nhiều cơ sở dữ liệu trong thế giới thực mà bạn gặp phải được thiết kế không đúng cách hoặc có nhiều điểm bất thường nếu chúng được sửa đổi không đúng cách theo thời gian. Điều đó có nghĩa là bạn có thể được yêu cầu thiết kế lại và sửa đổi cơ sở dữ liệu hiện có, trên thực tế, đây là những cái bẫy bất thường. Do đó, bạn nên lưu ý các nguyên tắc và quy trình thiết kế tốt cũng như các quy trình chuẩn hóa.

Đầu tiên, ERD được tạo thông qua một quy trình lặp đi lặp lại. Bạn bắt đầu bằng việc xác định các thực thể liên quan, các thuộc tính và mối quan hệ của chúng. Sau đó, bạn sử dụng kết quả để xác định các thực thể và thuộc tính bổ sung. ERD cung cấp bức tranh toàn cảnh hoặc cái nhìn vĩ mô về các yêu cầu và hoạt động dữ liệu của một tổ chức.

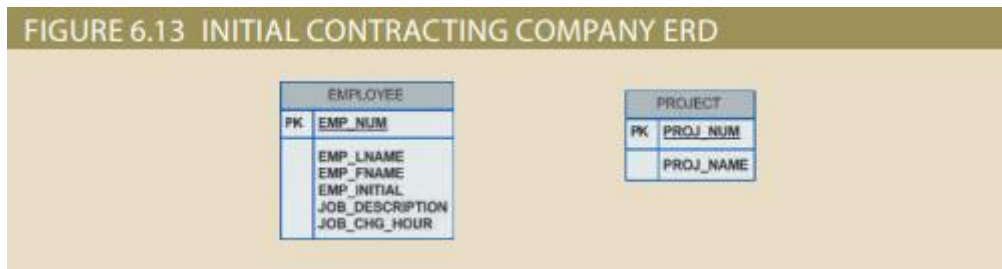
Thứ hai, việc chuẩn hóa tập trung vào các đặc điểm của các thực thể cụ thể; nghĩa là, chuẩn hóa thể hiện chế độ xem vi mô về các thực thể trong ERD. Ngoài ra, như bạn đã học trong các phần trước của chương này, quá trình chuẩn hóa có thể mang lại các thực thể và thuộc tính bổ sung được tích hợp vào ERD. Do đó, rất khó để tách chuẩn hóa khỏi mô hình ER; hai kỹ thuật này được sử dụng trong một quy trình lặp và tăng dần. Để hiểu vai trò thích hợp của việc chuẩn hóa trong quy trình thiết kế, bạn nên xem xét lại các hoạt động của công ty hợp đồng có các bảng đã được chuẩn hóa trong các phần trước. Những hoạt động đó có thể được tóm tắt bằng cách sử dụng các quy tắc nghiệp vụ sau:

- Công ty quản lý nhiều dự án.
- Mỗi dự án đều yêu cầu sự phục vụ của nhiều nhân viên.
- Một nhân viên có thể được phân công vào nhiều dự án khác nhau.
- Một số nhân viên không được phân công vào một dự án và thực hiện các nhiệm vụ không liên quan cụ thể đến một dự án. Một số nhân viên là thành viên của nhóm lao động được chia sẻ bởi tất cả các nhóm dự án. Ví dụ, thư ký điều hành của công ty sẽ không được giao phụ trách bất kỳ dự án cụ thể nào.
- Mỗi nhân viên có một phân loại công việc chính duy nhất để xác định mức thanh toán theo giờ.
- Nhiều nhân viên có thể có cùng một loại công việc. Ví dụ: công ty tuyển dụng nhiều kỹ sư điện.

Với mô tả đơn giản về hoạt động của công ty, hai thực thể và thuộc tính của chúng ban đầu được xác định:

- PROJECT (**PROJ_NUM** , PROJ_NAME)
- EMPLOYEE (**EMP_NUM** , EMP_LNAME, EMP_FNAME, EMP_INITIAL, JOB_DESCRIPTION, JOB_CHG_HOUR)

Hai thực thể đó tạo thành ERD ban đầu được hiển thị trong Hình 6.13.



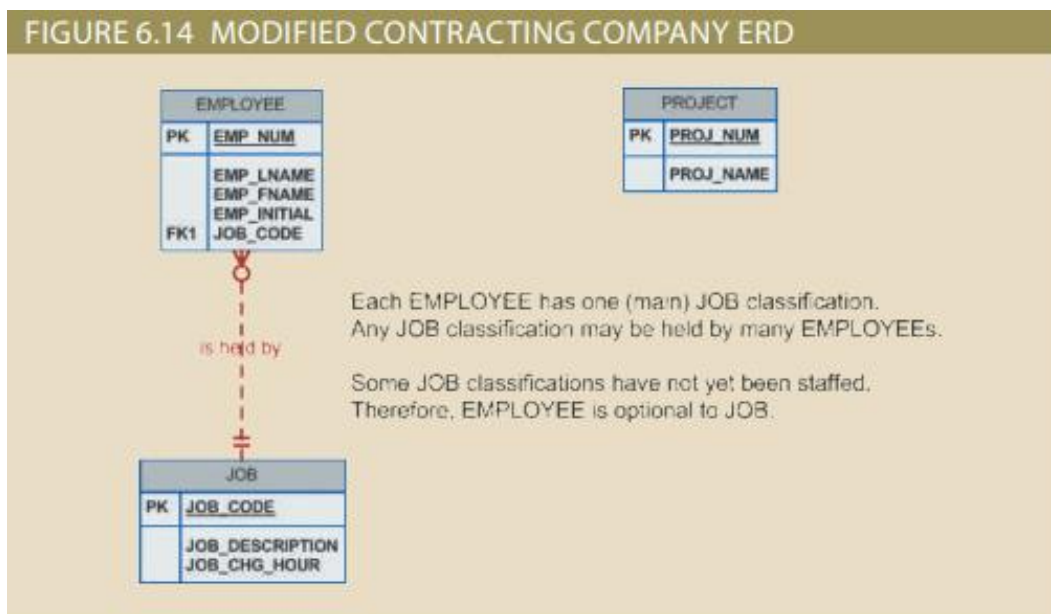
Sau khi tạo ERD ban đầu được hiển thị trong Hình 6.13, các dạng thông thường được xác định:

- **PROJECT** ở dạng 3NF và không cần sửa đổi tại thời điểm này.
- **EMPLOYEE** yêu cầu xem xét kỹ lưỡng hơn. Thuộc tính **JOB_DESCRIPTION** xác định các phân loại công việc như Systems Analyst, Database Designer, and Programmer (Nhà phân tích hệ thống, Nhà thiết kế cơ sở dữ liệu và Lập trình viên). Đổi lại, những phân loại đó sẽ xác định tỷ lệ thanh toán, **JOB_CHG_HOUR**. Do đó, **EMPLOYEE** chứa một phụ thuộc bắc cầu.

Việc loại bỏ phụ thuộc bắc cầu của **EMPLOYEE** mang lại ba thực thể:

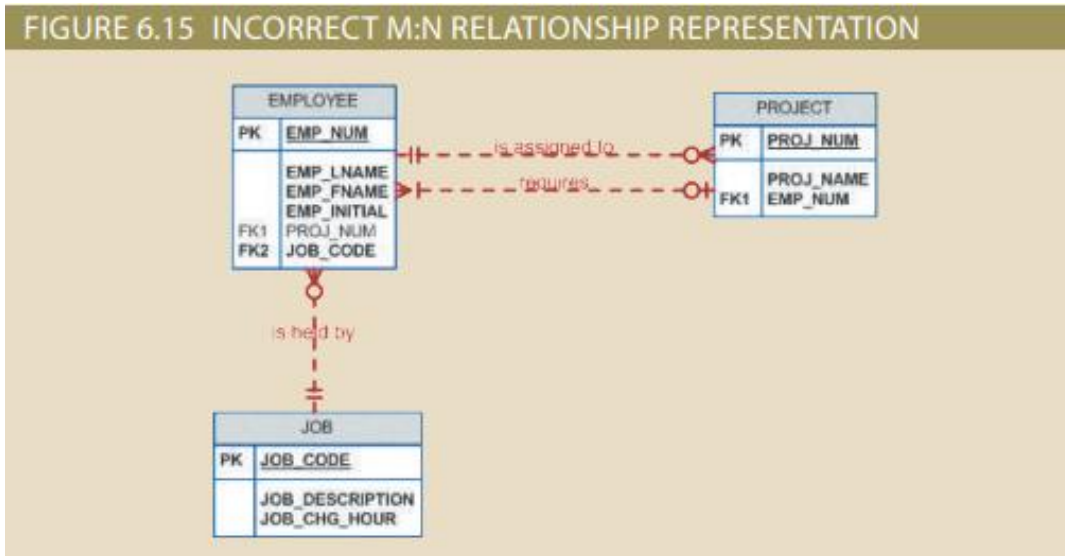
- **PROJECT** (**PROJ_NUM** , PROJ_NAME)
- **EMPLOYEE** (**EMP_NUM** , EMP_LNAME, EMP_FNAME, EMP_INITIAL, JOB_CODE)
- **JOB** (**JOB_CODE** , JOB_DESCRIPTION, JOB_CHG_HOUR)

Bởi vì quá trình chuẩn hóa mang lại một thực thể bổ sung (**JOB**), ERD ban đầu được sửa đổi như trong Hình 6.14.

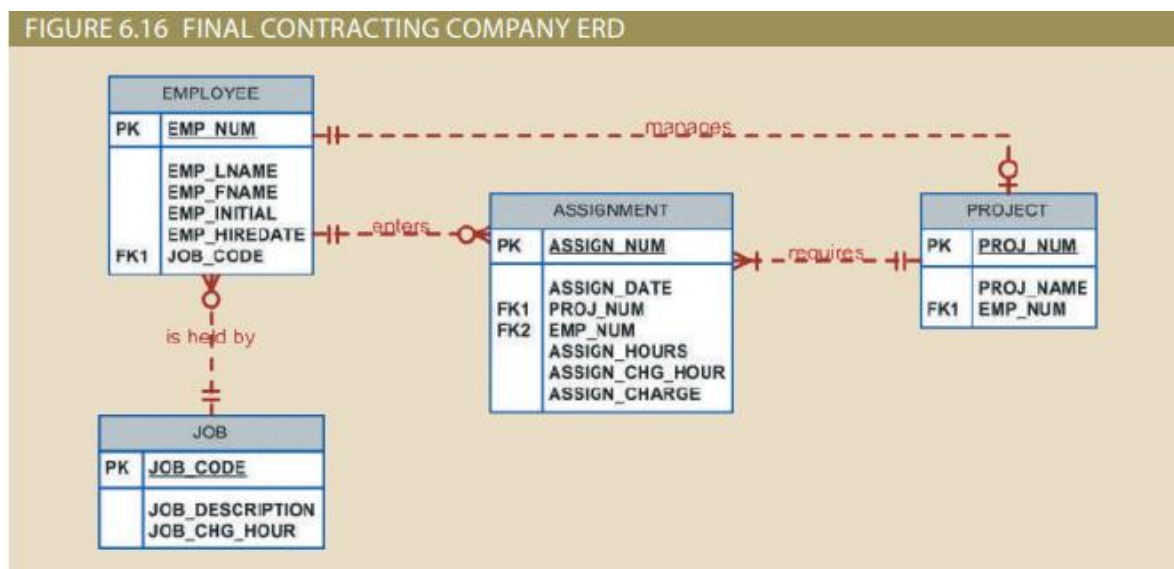


Để thể hiện mối quan hệ M:N giữa **EMPLOYEE** và **PROJECT**, bạn có thể nghĩ rằng có thể sử dụng hai mối quan hệ 1:M - một nhân viên có thể được chỉ định cho nhiều dự án và mỗi

dự án có thể có nhiều nhân viên được chỉ định cho nó. (Xem Hình 6.15.) Thật không may, cách biểu diễn đó tạo ra một thiết kế không thể triển khai chính xác.



Do không thể triển khai mối quan hệ M:N giữa EMPLOYEE và PROJECT, nên ERD trong Hình 6.15 phải được sửa đổi để bao gồm thực thể ASSIGNMENT để theo dõi việc phân công nhân viên cho các dự án, do đó mang lại ERD được hiển thị trong Hình 6.16. Thực thể ASSIGNMENT trong Hình 6.16 sử dụng các khóa chính từ các thực thể PROJECT và EMPLOYEE để làm khóa ngoại của nó. Tuy nhiên, hãy lưu ý rằng trong quá trình triển khai này, khóa chính thay thế của thực thể ASSIGNMENT là ASSIGN_NUM, để tránh sử dụng khóa chính tổng hợp. Do đó, mối quan hệ “enters” giữa EMPLOYEE và ASSIGNMENT cũng như mối quan hệ “requires” giữa PROJECT và ASSIGNMENT được thể hiện là yếu hoặc không xác định được.



Trong Hình 6.16, thuộc tính ASSIGN_HOURS được gán cho thực thể tổng hợp có tên ASSIGNMENT. Bởi vì bạn có thể sẽ cần thông tin chi tiết về người quản lý của từng dự án nên việc tạo mối quan hệ “manages” là rất hữu ích. Mối quan hệ “manages” được thực

hiện thông qua khóa ngoại trong PROJECT. Cuối cùng, một số thuộc tính bổ sung có thể được tạo ra để cải thiện khả năng của hệ thống trong việc tạo ra thông tin bổ sung. Ví dụ: bạn có thể muốn bao gồm ngày nhân viên được thuê (EMP_HIREDATE) để theo dõi tuổi thọ của nhân viên. Dựa trên sửa đổi cuối cùng này, mô hình sẽ bao gồm bốn thực thể và các thuộc tính của chúng:

PROJECT (**PROJ_NUM** , PROJ_NAME, EMP_NUM)

EMPLOYEE (**EMP_NUM** , EMP_LNAME, EMP_FNAME, EMP_INITIAL, EMP_HIREDATE, JOB_CODE)

JOB (**JOB_CODE** , JOB_DESCRIPTION, JOB_CHG_HOUR)

ASSIGNMENT (**ASSIGN_NUM** , ASSIGN_DATE, PROJ_NUM, EMP_NUM, ASSIGN_HOURS, ASSIGN_CHG_HOUR, ASSIGN_CHARGE)

Quá trình thiết kế hiện đang đi đúng hướng. ERD thể hiện các hoạt động một cách chính xác và các thực thể hiện phản ánh sự phù hợp của chúng với 3NF. Sự kết hợp giữa chuẩn hóa và mô hình ER mang lại một ERD hữu ích, mà các thực thể của nó hiện có thể được chuyển thành các cấu trúc bảng thích hợp. Trong Hình 6.15, lưu ý rằng PROJECT là tùy chọn đối với EMPLOYEE trong mối quan hệ “manages”. Tùy chọn này tồn tại vì không phải tất cả nhân viên đều quản lý dự án. Nội dung cơ sở dữ liệu cuối cùng được hiển thị trong Hình 6.17.

FIGURE 6.17 THE IMPLEMENTED DATABASE

Table name: EMPLOYEE

EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_HIREDATE	JOB_CODE
101	News	John	G	06-Nov-00	502
102	Senior	David	H	12-Jul-89	501
103	Arbough	June	E	01-Dec-97	503
104	Ramosas	Anna	K	15-Nov-86	501
105	Johnson	Alice	K	01-Feb-94	502
106	Smithfield	Williams		22-Jun-05	500
107	Alenze	Maria	D	10-Oct-94	500
108	Washington	Ralph	B	22-Aug-89	501
109	Smith	Larry	W	19-Jul-99	501
110	Olenka	Gerald	A	11-Dec-96	505
111	Watbath	Geoff	B	04-Apr-89	506
112	Smithson	Darlene	M	23-Oct-95	507
113	Joerbood	Delbert	K	15-Nov-94	508
114	Jones	Annelise		20-Aug-91	508
115	Bawang	Travis	B	25-Jan-90	501
116	Pratt	Gerald	L	05-Mar-95	510
117	Williamson	Angie	H	19-Jun-94	509
118	Fremmer	James	J	04-Jan-06	510

Database name: Ch06_ConstructCo

Table name: JOB

JOB_CODE	JOB_DESCRIPTION	JOB_CHG_HOUR
500	Programmer	35.75
501	Systems Analyst	96.75
502	Database Designer	105.00
503	Electrical Engineer	84.50
504	Mechanical Engineer	67.90
505	Civil Engineer	55.78
506	Clinical Support	26.87
507	DSS Analyst	45.95
508	Applications Designer	48.10
509	Bio Technician	34.55
510	General Support	18.36

Table name: PROJECT

PROJ_NUM	PROJ_NAME	EMP_NUM
15	Evergreen	105
18	Amber Wave	104
22	Rolling Tide	113
25	Starlight	101

Table name: ASSIGNMENT

ASSIGN_NUM	ASSIGN_DATE	PROJ_NUM	EMP_NUM	ASSIGN_HOURS	ASSIGN_CHG_HOUR	ASSIGN_CHARGE
1001	04-Mar-18	15	103	2.6	84.50	219.70
1002	04-Mar-18	18	118	1.4	18.36	25.70
1003	05-Mar-18	15	101	3.6	105.00	378.00
1004	05-Mar-18	22	113	2.5	48.10	120.25
1005	05-Mar-18	15	103	1.9	84.50	160.55
1006	05-Mar-18	25	115	4.2	96.75	406.35
1007	05-Mar-18	22	105	5.2	105.00	546.00
1008	05-Mar-18	25	101	1.7	105.00	178.50
1009	05-Mar-18	15	105	2.0	105.00	210.00
1010	06-Mar-18	15	102	3.8	96.75	367.65
1011	06-Mar-18	22	104	2.6	96.75	251.55
1012	06-Mar-18	15	101	2.3	105.00	241.50
1013	06-Mar-18	25	114	1.8	48.10	86.58
1014	06-Mar-18	22	111	4.0	26.87	107.48
1015	06-Mar-18	25	114	3.4	48.10	163.54
1016	06-Mar-18	18	112	1.2	45.95	55.14
1017	06-Mar-18	18	118	2.0	18.36	36.72
1018	06-Mar-18	18	104	2.6	96.75	251.55
1019	06-Mar-18	15	103	3.0	84.50	253.50
1020	07-Mar-18	22	105	2.7	105.00	283.50
1021	08-Mar-18	25	108	4.2	96.75	406.35
1022	07-Mar-18	25	114	5.8	48.10	278.98
1023	07-Mar-18	22	106	2.4	35.75	85.80

6-8. Không chuẩn hóa

Điều quan trọng cần nhớ là việc triển khai cơ sở dữ liệu quan hệ tối ưu yêu cầu tất cả các bảng ít nhất phải ở dạng 3NF. Một DBMS quan hệ tốt sẽ vượt trội trong việc quản lý các quan hệ đã chuẩn hóa - nghĩa là, các quan hệ không có bất kỳ sự dư thừa không cần thiết nào có thể gây ra các dị thường về dữ liệu. Mặc dù việc tạo ra các quan hệ chuẩn hóa là một mục tiêu thiết kế cơ sở dữ liệu quan trọng nhưng nó chỉ là một trong nhiều mục tiêu như vậy. Thiết kế cơ sở dữ liệu tốt cũng xem xét các yêu cầu xử lý (hoặc báo cáo) và tốc độ xử lý. Vấn đề với việc chuẩn hóa là khi các bảng được phân tách để phù hợp với các yêu cầu chuẩn hóa, số lượng bảng cơ sở dữ liệu sẽ mở rộng. Vì vậy, để tạo ra thông tin, dữ liệu phải được tập hợp từ nhiều bảng khác nhau. Việc kết hợp một số lượng lớn các bảng yêu cầu các hoạt động đầu vào/đầu ra (I/O) bổ sung và logic xử lý, do đó làm giảm tốc độ hệ thống. Hầu hết các hệ thống cơ sở dữ liệu quan hệ đều có thể xử lý các phép nối rất hiệu quả. Tuy nhiên, trong một số trường hợp hiếm gặp và không thường xuyên có thể cho phép một số mức độ không chuẩn hóa, do đó tốc độ xử lý có thể được tăng lên.

Hãy nhớ rằng lợi thế của tốc độ xử lý cao hơn phải được cân nhắc cẩn thận trước nhược điểm của dữ liệu sự bất thường. Mặt khác, một số dữ liệu thường chỉ có giá trị về mặt lý thuyết. Ví dụ: mọi người trong môi trường cơ sở dữ liệu thế giới thực có nên lo lắng rằng ZIP_CODE xác định CITY trong bảng CUSTOMER có khóa chính là mã số khách hàng không? Việc tạo một bảng riêng cho

ZIP (**ZIP CODE** , CITY)

để loại bỏ sự phụ thuộc bắc cầu khỏi bảng CUSTOMER có thực sự thiết thực không? (Có lẽ câu trả lời của bạn cho câu hỏi đó sẽ thay đổi nếu bạn đang làm công việc tạo danh sách gửi thư.) Như đã giải thích trước đó, vấn đề với các quan hệ không chuẩn hóa và dữ liệu dư thừa là tính toàn vẹn của dữ liệu có thể bị xâm phạm do khả năng chen, cập nhật và xóa các bất thường. Lời khuyên rất đơn giản: hãy sử dụng lẽ thường trong quá trình chuẩn hóa.

Hơn nữa, trong một số trường hợp, quy trình thiết kế cơ sở dữ liệu có thể đưa ra một số dữ liệu dư thừa ở mức độ nhỏ trong mô hình, như đã thấy trong ví dụ trước. Trên thực tế, điều này tạo ra các mối quan hệ “không chuẩn hóa”. Bảng 6.6 cho thấy một số ví dụ phổ biến về dư thừa dữ liệu thường thấy trong triển khai cơ sở dữ liệu.

Một ví dụ toàn diện hơn về nhu cầu không chuẩn hóa do yêu cầu báo cáo là trường hợp báo cáo đánh giá giảng viên trong đó mỗi hàng liệt kê điểm đạt được trong bốn học kỳ vừa qua được giảng dạy. (Xem Hình 6.18.)

Mặc dù báo cáo này có vẻ đơn giản nhưng vấn đề là dữ liệu được lưu trữ trong một bảng không chuẩn hóa, trong đó mỗi hàng thể hiện một điểm khác nhau của một giảng viên nhất định trong một học kỳ nhất định. (Xem Hình 6.19.)

Khó khăn trong việc chuyển đổi dữ liệu nhiều hàng thành dữ liệu nhiều cột còn phức tạp hơn bởi thực tế là bốn học kỳ cuối cùng được giảng dạy không nhất thiết giống nhau đối với tất cả giảng viên. Một số có thể đã nghỉ phép, một số có thể đã được bổ nhiệm nghiên cứu, một số có thể là giảng viên mới chỉ làm việc có hai học kỳ, v.v. Để tạo báo cáo này, hai bảng trong Hình 6.18 đã được sử dụng. Bảng EVALDATA là bảng dữ liệu tổng thể chứa điểm đánh giá của từng giảng viên cho từng học kỳ giảng dạy; bảng này được chuẩn hóa. Bảng FACHIST chứa bốn điểm dữ liệu cuối cùng - tức là điểm đánh giá và học kỳ - cho mỗi giảng viên. Bảng FACHIST là một bảng không chuẩn hóa tạm thời được tạo từ bảng EVALDATA thông qua một loạt các truy vấn. (Bảng FACHIST là cơ sở cho báo cáo được hiển thị trong Hình 6.18.)

TABLE 6.6		
COMMON DENORMALIZATION EXAMPLES		
CASE (trường hợp)	EXAMPLE (ví dụ)	RATION AND CONTROLS (tỉ lệ và kiểm soát)
Dữ liệu dự phòng	Lưu thuộc tính ZIP và CITY trong bảng AGENT khi ZIP xác định CITY (xem Hình 2.2)	Tránh các hoạt động tham gia bổ sung Chương trình có thể xác thực thành phố (hộp thả xuống) dựa trên mã zip
Dữ liệu có nguồn gốc	Lưu trữ STU_HRS và STU_CLASS (phân loại sinh viên) khi STU_HRS xác định STU_CLASS (xem Hình 3.28)	Tránh các hoạt động tham gia bổ sung Chương trình có thể xác thực phân loại (tra cứu) dựa trên giờ của sinh viên
Dữ liệu tổng hợp trước (cũng là dữ liệu phái sinh)	Lưu trữ giá trị tổng hợp điểm trung bình (STU_GPA) của học sinh trong bảng SINH VIÊN khi giá trị này có thể được tính từ các bảng ENROLL and COURSE (xem Hình 3.28)	Tránh các hoạt động tham gia bổ sung Chương trình tính toán GPA mỗi khi nhập hoặc cập nhật điểm STU_GPA chỉ có thể được cập nhật thông qua thủ tục quản trị
Yêu cầu thông tin	Sử dụng bảng không chuẩn hóa tạm thời để giữ dữ liệu báo cáo; điều này là bắt buộc khi tạo báo cáo bằng trong đó các cột biểu thị dữ liệu được lưu trữ trong bảng dưới dạng hàng (xem Hình 6.17 và 6.18)	Không thể tạo dữ liệu theo yêu cầu của báo cáo bằng cách sử dụng SQL đơn giản Không cần duy trì bảng Bảng tạm thời sẽ bị xóa sau khi báo cáo hoàn tất Tốc độ xử lý không phải là vấn đề

FIGURE 6.18 THE FACULTY EVALUATION REPORT

Faculty Evaluation Report										
Instructor	Department	I		II		III		IV		Last Two Sem. Avg.
		Semester	Mean	Semester	Mean	Semester	Mean	Semester	Mean	
Alton	INFS	2017S	2.91	2016F	2.84	2016S	2.55	2015F	2.51	2.875
Ames	INFS	2017S	3.24	2016F	3.26	2016S	3.31	2015F	3.19	3.250
Crandon	INFS	2017S	3.93	2016F	3.95	2016S	3.91	2015F	3.88	3.940
Dumas	MGMT	2016F	3.66	2016S	3.69	2015F	3.56	2015S	3.72	3.675
Landon	BMOM	2017S	3.57	2016F	3.64	2016S	3.39	2015F	3.57	3.605
Lohar	ECON	2011F	3.53	2010F	3.53					3.530
Rohman	INFS	2008S	3.50							3.500

FIGURE 6.19 THE EVALDATA AND FACHIST TABLES

Table name: EVALDATA

ID	INSTRUCTOR	DEPARTMENT	MEAN	SEMESTER
3079	Allen	MFIS	2.79	2013F
3204	Allen	MFIS	2.86	2012S
3238	Allen	MFIS	2.88	2013F
3472	Allen	MFIS	2.72	2013S
3739	Allen	MFIS	2.2	2014F
3794	Allen	MFIS	2.69	2014S
3875	Allen	MFIS	2.54	2015F
4172	Allen	MFIS	2.12	2015S
4459	Allen	MFIS	2.84	2016F
4323	Allen	MFIS	2.68	2016S
4571	Allen	MFIS	2.98	2017S
3071	Ames	MFIS	3.35	2012F
3225	Ames	MFIS	2.82	2012S
3337	Ames	MFIS	3.24	2013F
3473	Ames	MFIS	2.79	2013S
3688	Ames	MFIS	3.28	2014F
3765	Ames	MFIS	3.24	2014S
3876	Ames	MFIS	3.19	2015F
4155	Ames	MFIS	2.59	2015S
4414	Ames	MFIS	3.26	2016F
4389	Ames	MFIS	3.24	2016S
4572	Ames	MFIS	3.24	2017S
3613	Grandin	MFIS	3.79	2014F
3077	Grandin	MFIS	3.88	2015F
4040	Grandin	MFIS	3.79	2015S
4248	Grandin	MFIS	3.86	2016F
4188	Grandin	MFIS	3.98	2016S
4573	Grandin	MFIS	3.80	2017S
2052	Dumas	MGMT	3.67	2008F
2216	Dumas	MGMT	3.66	2008S
2333	Dumas	MGMT	3.8	2009F

Table name: FACHIST

ID	INSTRUCTOR	DEPARTMENT	LAST1SEM	LAST2SEM	LAST3SEM	LAST4SEM	LAST5SEM
59602	Allen	MFIS	2017S	2.98	2016F	2.84	2016S
59603	Ames	MFIS	2017S	3.24	2016F	3.26	2016S
59605	Grandin	MFIS	2017S	3.93	2016F	3.95	2016S
59607	Dumas	MGMT	2016F	3.66	2016S	3.89	2015F
59608	London	EBOM	2017S	3.57	2016F	3.64	2016S
59610	Lohar	EBOM	2011F	3.53	2010F	3.53	
59611	Robson	MFIS	2008S	3.5			

Database name: Ch06_EVAL

Denormalized

Repeating Group

Normalized

Như được hiển thị trong báo cáo đánh giá của khoa, những xung đột giữa hiệu quả thiết kế, yêu cầu thông tin và hiệu suất thường được giải quyết thông qua các thỏa hiệp có thể bao gồm việc không chuẩn hóa. Trong trường hợp này, và giả sử có đủ không gian lưu trữ, các lựa chọn của người thiết kế có thể được thu hẹp lại thành:

- Lưu trữ dữ liệu trong một bảng không chuẩn hóa vĩnh viễn. Đây không phải là giải pháp được đề xuất vì bảng không chuẩn hóa có thể có dữ liệu bất thường (chèn, cập nhật và xóa). Giải pháp này chỉ khả thi nếu hiệu suất là một vấn đề.
- Tạo một bảng không chuẩn hóa tạm thời từ (các) bảng đã chuẩn hóa vĩnh viễn. Bảng không chuẩn hóa chỉ tồn tại trong thời gian cần thiết để tạo báo cáo; nó biến mất sau khi báo cáo được tạo ra. Do đó, không có vấn đề bất thường về dữ liệu. Giải pháp này chỉ thực tế nếu hiệu suất không phải là vấn đề và không có tùy chọn xử lý khả thi nào khác.

Như đã trình bày, *độ tinh khiết chuẩn hóa thường khó duy trì trong môi trường cơ sở dữ liệu hiện đại*. Bạn sẽ học trong Chương 13, Nghiệp vụ thông minh và Kho dữ liệu, rằng các dạng chuẩn hóa thấp hơn xuất hiện (và thậm chí là bắt buộc) trong các cơ sở dữ liệu chuyên biệt được gọi là kho dữ liệu. Những cơ sở dữ liệu chuyên biệt như vậy phản ánh nhu cầu ngày càng tăng về phạm vi và chiều sâu lớn hơn của dữ liệu mà các hệ thống hỗ trợ quyết định ngày càng dựa vào. Bạn sẽ phát hiện ra rằng kho dữ liệu thường xuyên sử dụng cấu trúc 2NF trong môi trường dữ liệu phức tạp, đa cấp, đa nguồn. Mặc dù việc chuẩn hóa là rất quan trọng, đặc biệt là trong cái gọi là môi trường cơ sở dữ liệu sản xuất, nhưng 2NF không còn bị coi nhẹ như trước nữa.

Mặc dù các bảng 2NF không phải lúc nào cũng có thể tránh được, nhưng vấn đề khi làm việc với các bảng có chứa các phụ thuộc một phần và/hoặc bắc cầu trong cơ sở dữ liệu sản xuất môi trường không nên được giảm thiểu. Bên cạnh khả năng tạo ra các dữ liệu bất thường gây rắc rối, các bảng không chuẩn hóa trong cơ sở dữ liệu sản xuất có xu hướng mắc phải các lỗi sau:

- Cập nhật dữ liệu kém hiệu quả hơn vì các chương trình đọc và cập nhật bảng phải xử lý các bảng lớn hơn.

- Việc lập chỉ mục phức tạp hơn. Đơn giản là không thực tế khi xây dựng tất cả các chỉ mục cần thiết cho nhiều thuộc tính có thể nằm trong một bảng không chuẩn hóa duy nhất.
- Các bảng không chuẩn hóa không mang lại chiến lược đơn giản nào để tạo các bảng ảo được gọi là *các khung nhìn*. Bạn sẽ học cách tạo và sử dụng các dạng xem trong Chương 8, SQL nâng cao.

Hãy nhớ rằng không thể tạo ra thiết kế tốt trong các chương trình ứng dụng sử dụng cơ sở dữ liệu. Ngoài ra, hãy nhớ rằng các bảng cơ sở dữ liệu không chuẩn hóa thường dẫn đến nhiều thảm họa dư thừa dữ liệu trong cơ sở dữ liệu sản xuất, chẳng hạn như các vấn đề đã được kiểm tra cho đến nay. Nói cách khác, hãy sử dụng tính năng không chuẩn hóa một cách thận trọng và đảm bảo rằng bạn có thể giải thích tại sao các bảng không chuẩn hóa lại là lựa chọn tốt hơn trong một số trường hợp so với các bảng được chuẩn hóa tương ứng của chúng.

6-9. Danh sách kiểm tra mô hình hóa dữ liệu

Trong các chương của Phần 2, bạn đã tìm hiểu cách mô hình hóa dữ liệu chuyển một môi trường thế giới thực cụ thể thành mô hình dữ liệu đại diện cho dữ liệu, người dùng, quy trình và tương tác trong thế giới thực. Các kỹ thuật lập mô hình mà bạn đã học cho đến nay sẽ cung cấp cho bạn những công cụ cần thiết để tạo ra các thiết kế cơ sở dữ liệu thành công. Tuy nhiên, cũng giống như bất kỳ phi công giỏi nào đều sử dụng danh sách kiểm tra để đảm bảo rằng tất cả đều chuẩn bị cho chuyến bay thành công, danh sách kiểm tra lập mô hình dữ liệu được hiển thị trong Bảng 6.7 sẽ giúp đảm bảo rằng bạn thực hiện thành công các nhiệm vụ lập mô hình dữ liệu dựa trên các khái niệm và công cụ bạn đã học trong văn bản này.

Lưu ý:

Bạn cũng có thể tìm thấy danh sách kiểm tra mô hình hóa dữ liệu này ở bìa trước bên trong của cuốn sách này để dễ dàng tham khảo.

TABLE 6.7
DATA-MODELING CHECKLIST
QUY TẮC NGHIỆP VỤ
• Lập tài liệu và xác minh đúng cách tất cả các quy tắc nghiệp vụ với người dùng cuối. • Đảm bảo rằng tất cả các quy tắc nghiệp vụ được viết chính xác, rõ ràng và đơn giản. Các quy tắc nghiệp vụ phải giúp xác định các thực thể, thuộc tính, mối quan hệ và các ràng buộc. • Xác định nguồn của tất cả các quy tắc nghiệp vụ và đảm bảo rằng mỗi quy tắc nghiệp vụ đều hợp lý, ghi ngày tháng và được cơ quan phê duyệt phê duyệt.
Mô hình hóa dữ liệu

Quy ước đặt tên: Tất cả các tên phải được giới hạn về độ dài (kích thước phụ thuộc vào cơ sở dữ liệu). • Tên thực thể: • Phải là danh từ quen thuộc với doanh nghiệp, ngắn gọn và có ý nghĩa • Nên ghi lại các chữ viết tắt, từ đồng nghĩa và bí danh cho từng thực thể • Phải là duy nhất trong mô hình • Đối với các thực thể tổng hợp, có thể bao gồm sự kết hợp của các tên viết tắt của các thực thể được liên kết thông qua thực thể tổng hợp • Tên thuộc tính: • Phải là duy nhất trong thực thể • Nên sử dụng tên viết tắt của thực thể làm tiền tố • Nên mô tả đặc tính • Nên sử dụng các hậu tố như _ID, _NUM hoặc _CODE cho PK thuộc tính • Không được là từ dành riêng • Không được chứa dấu cách hoặc ký tự đặc biệt như @, !, hoặc & • Tên mối quan hệ: • Phải là động từ chủ động hoặc bị động chỉ rõ bản chất của mối quan hệ
Các thực thể: • Mỗi thực thể nên đại diện cho một chủ thể duy nhất. • Mỗi thực thể nên đại diện cho một tập hợp các thực thể có thể phân biệt được. • Tất cả các thực thể phải ở mức 3NF hoặc cao hơn. Bất kỳ thực thể nào dưới 3NF đều phải hợp lý. • Độ chi tiết của thực thể thực thể cần được xác định rõ ràng. • PK phải được xác định rõ ràng và hỗ trợ mức độ chi tiết của dữ liệu đã chọn.
Thuộc tính: • Phải đơn giản và có một giá trị (dữ liệu nguyên tử) • Nên ghi lại các giá trị mặc định, ràng buộc, từ đồng nghĩa và bí danh • Thuộc tính phái sinh phải được xác định rõ ràng và bao gồm (các) nguồn • Không được dư thừa trừ khi điều này là bắt buộc đối với giao dịch độ chính xác, hiệu suất hoặc duy trì lịch sử • Thuộc tính không khóa phải phụ thuộc hoàn toàn vào thuộc tính PK
Các mối quan hệ: • Nên xác định rõ ràng những người tham gia mối quan hệ • Nên xác định rõ ràng sự tham gia, kết nối và tính chính yếu của tài liệu
Mô hình ER: • Nên được xác thực dựa trên các quy trình dự kiến: chèn, cập nhật và xóa • Nên đánh giá vị trí, thời điểm và cách duy trì lịch sử • Không nên chứa các mối quan hệ dư thừa trừ khi được yêu cầu (xem thuộc tính) • Nên giảm thiểu sự dư thừa dữ liệu để đảm bảo cập nhật tại một nơi duy nhất • Phải tuân theo quy tắc dữ liệu tối thiểu: Tất cả những gì cần thiết đều có ở đó và tất cả những gì ở đó là cần thiết.

Bản tóm tắt:

- Chuẩn hóa là một kỹ thuật được sử dụng để thiết kế các bảng trong đó sự dư thừa dữ liệu được giảm thiểu. Ba dạng chuẩn đầu tiên (1NF, 2NF và 3NF) là phổ biến nhất. Từ quan điểm cấu trúc, các dạng chuẩn cao hơn sẽ tốt hơn dạng chuẩn thấp vì dạng chuẩn cao hơn mang lại tương đối ít dữ liệu dư thừa trong cơ sở dữ liệu hơn. Hầu hết tất cả các thiết kế nghiệp vụ đều sử dụng 3NF làm dạng chuẩn lý tưởng. Một dạng 3NF đặc biệt, hạn chế hơn được gọi là dạng chuẩn Boyce-Codd, hay BCNF, cũng được sử dụng.
- Một bảng ở dạng 1NF khi tất cả các thuộc tính khóa được xác định và tất cả các thuộc tính còn lại đều phụ thuộc vào khóa chính. Tuy nhiên, một bảng trong 1NF vẫn có thể chứa cả phụ thuộc một phần và phụ thuộc bắc cầu. Sự phụ thuộc một phần là sự phụ thuộc trong đó một thuộc tính chỉ phụ thuộc về một chức năng vào một phần của khóa chính đa thuộc tính. Sự phụ thuộc bắc cầu là sự phụ thuộc trong đó một thuộc tính phụ thuộc về một chức năng vào một thuộc tính không khóa khác. Một bảng có khóa chính thuộc tính đơn không thể biểu thị sự phụ thuộc một phần.
- Một bảng ở dạng 2NF khi nó ở dạng 1NF và không chứa các phụ thuộc một phần. Do đó, bảng 1NF sẽ tự động ở dạng 2NF khi khóa chính của nó chỉ dựa trên một thuộc tính duy nhất. Một bảng ở 2NF vẫn có thể chứa các phụ thuộc bắc cầu.
- Một bảng ở trạng thái 3NF khi nó ở trạng thái 2NF và không chứa các phụ thuộc bắc cầu. Với định nghĩa đó, dạng chuẩn Boyce-Codd (BCNF) chỉ là một trường hợp 3NF đặc biệt trong đó tất cả các khóa xác định là các khóa ứng viên. Khi một bảng chỉ có một khóa ứng cử viên duy nhất, bảng 3NF sẽ tự động ở BCNF.
- Một bảng không thuộc 3NF có thể được chia thành các bảng mới cho đến khi tất cả các bảng đáp ứng yêu cầu 3NF.
- Chuẩn hóa là một phần quan trọng - nhưng chỉ là một phần - của quá trình thiết kế. Vì các thực thể và thuộc tính được xác định trong quá trình lập mô hình ER, nên mỗi thực thể (bộ) phải kiểm tra chuẩn hóa và hình thành các thực thể (bộ) mới theo yêu cầu. Kết hợp các thực thể đã chuẩn hóa vào ERD và tiếp tục quy trình ER lặp lại cho đến khi tất cả các thực thể và thuộc tính của chúng được xác định và tất cả các bảng tương đương đều ở dạng 3NF.
- Một bảng trong 3NF có thể chứa các phụ thuộc đa giá trị tạo ra nhiều giá trị null hoặc dữ liệu dư thừa. Do đó, có thể cần phải chuyển đổi bảng 3NF sang dạng chuẩn thứ tư (4NF) bằng cách tách bảng để loại bỏ các phụ thuộc đa giá trị. Do đó, một bảng ở dạng 4NF khi nó ở dạng 3NF và không chứa các phụ thuộc đa giá trị.
- Số lượng bảng càng lớn thì càng cần nhiều thao tác I/O và logic xử lý bổ sung để nối chúng. Do đó, các bảng đôi khi không được chuẩn hóa thành I/O không mang lại năng suất để tăng tốc độ xử lý. Thật không may, với các bảng lớn hơn, bạn phải trả giá cho tốc độ xử lý tăng lên bằng cách làm cho việc cập nhật dữ liệu kém hiệu quả hơn, làm cho việc lập chỉ mục trở nên cồng kềnh hơn và bằng cách đưa ra các dữ liệu dư thừa có khả năng tạo ra các dị thường về dữ liệu. Trong thiết kế cơ sở dữ

liệu sản xuất, hãy sử dụng tính năng không chuẩn hóa một cách tiết kiệm và thận trọng.

- Danh sách kiểm tra mô hình hóa dữ liệu cung cấp một cách để người thiết kế kiểm tra xem ERD có đáp ứng một bộ yêu cầu tối thiểu hay không.

Key Terms		
atomic attribute	first normal form (1NF)	partial dependency
atomicity	fourth normal form (4NF)	prime attribute
Boyce-Codd normal form (BCNF)	granularity	repeating group
denormalization	key attribute	second normal form (2NF)
dependency diagram	nonkey attribute	third normal form (3NF)
determinant	nonprime attribute	transitive dependency
	normalization	unnormalized data